

Machine Learning Techniques for Matching Candidates' Profiles With Job Advertisements

Aleksandra Davidova Stefanova

**Master Programme in Data Science
2025**

Luleå University of Technology
Department of Computer Science, Electrical and Space Engineering

[This page intentionally left blank]

Abstract

The current study presents 3 different family types and 11 distinct models for resume-job matching tasks. The aim is to make a direct comparison between the machine learning models and traditional keyword matching algorithms. TF-IDF techniques and 10 different types of transformer-based models are used to match resumes to their corresponding job ads using a public dataset from Kaggle and a private corporate dataset provided by the Swedish MindDig company. Results showed that the Ada and T5-based models generated the highest cosine similarity scores, whereas the TF-IDF performed very poorly due to the model's inability to capture semantic similarity in text. This suggests that traditional keyword matching algorithms are not well-suited for resume-job matching, unlike modern embedding ML models.

Acknowledgements

I would like to thank my supervisor, Oluwatosin Adewumi, for his guidance and assistance during the extensive process of writing this thesis. Without his help, this research would have not been possible. I would also like to express my gratitude to the Luleå University of Technology program coordinators and teachers for giving me the opportunity to be a part of this program. It has been an incredible experience for me, which has given me the chance to extend my knowledge in the amazing world of data science. I would also like to thank my family and friends for their continuous encouragement and support. Lastly, I would like to express my appreciation towards the company MindDig for providing the extensive dataset files, which have been of tremendous help during the process of building the models.

Internal supervisor: Oluwatosin Adewumi
External supervisor: Tobias Eriksson

Keywords

Job-Resume Matching, Machine-Learning, Embeddings, Sentence Transformer, Cosine Similarity, TF-IDF

1. Introduction	5
1.1 Introduction and Research Setting	5
1.2 Purpose Statement and Research Questions	5
1.3 Topic Justification	6
1.4 Scope and Limitations	6
1.5 Knowledge Gap	7
1.6 Ethical Considerations and Bias	7
2. Review of the Literature	7
2.1 Search strategy	7
2.2 Decision Tree-Based Models	8
2.3 Classical Machine Learning Models	8
2.4 Deep Learning Models (CNNs, RNNs, BiLSTM, Attention)	9
2.5 Transformer-Based and LLM Models	11
2.6 Hybrid & Multimodal Systems	14
2.7 Embedding and NLP-Based Models	16
3. Methodology	16
3.1 Datasets	16
3.2 Data preprocessing	17
3.3 Models	17
3.3.1 Semantic embedding model + cosine similarities	18
3.3.2 SentenceTransformer + cosine similarities	19
3.3.2.1 all-MiniLM-L6-v2	20
3.3.2.2 all-mpnet-base-v2	20
3.3.2.3 all-MiniLM-L12-v2	20
3.3.2.4 multi-qa-MiniLM-L6-cos-v1	21
3.3.2.5 multi-qa-distilbert-cos-v1	21
3.3.2.6 paraphrase-mpnet-base-v2	21
3.3.2.7 sentence-t5-base	21
3.3.2.8 all-distilroberta-v1	22
3.3.2.9 gtr-t5-base	22
3.3.2.10 Cosine similarity	23
3.3.3 TF-IDF + cosine similarities	23
3.3.4 Why the above models were chosen for this research	24
3.4 Train/Test/Validation Splits	25
Dataset train/test/validation split for TF-IDF	25
Dataset train/test/validation split for the Semantic embedding model	25
Dataset train/test/validation split for Sentence Transformers	25
3.5 Evaluation runs	25
3.6 Similarity threshold	26
3.7 Method limitations	26
4. Results	27
4.1 Result summary	27

4.2 Qualitative results (correct prediction examples)	28
4.3 Error analysis (incorrect prediction examples)	28
5. Discussion	29
5.1 Interpretation of Results	29
5.1.1 High-performance models	29
5.1.2 Moderate performance models	31
5.1.3 Low-performance models	31
5.1.4 Very Low performance models	31
5.2 Future research	32
5.3 Strengths and Limitations of the current study	33
6. Conclusion	33
7. ChatGPT	34
References	34
Appendices	39
Appendix A - Dataset categories	39
Appendix B - Distribution of similarity scores	40
Kaggle dataset	40
Appendix C - Average similarity scores across runs	44
Kaggle dataset	44
Appendix D - Qualitative results (correct prediction examples)	49
Appendix E - Error analysis (incorrect prediction examples)	54

1. Introduction

1.1 Introduction and Research Setting

With the ever-increasing growth of online recruitment data, job resume matching has become a central task for modern organisations. The growing amount of online recruitment data makes conventional manual selection cumbersome and highly ineffective. It also emerges from the literature that employers just glance through a resume for 6-7 seconds at most, which is a relatively small amount of time that can hardly provide a comprehensive review (Dharmendra et al., 2025). Therefore, there is an increasing demand for effective automatic screening systems to match jobs with suitable resumes and accelerate the recruitment process. This task is typically handled by supervised text matching techniques. It has been estimated that between 60% and 75% of the resumes never get through the ATS before even reaching a recruiter (Jha et al., 2025). Traditional keyword matching algorithms, which were applied in the early days, were only partially effective as they often failed to capture the overall context of resumes and struggled to interpret CVs when the exact job advertisement keywords were absent from the resumes (Gautam et al., 2025). This is why these types of systems often fail to attract highly qualified individuals whose CVs do not contain specific keywords. Due to the restrictions of traditional manual and automotive screening systems, machine learning techniques are gaining increasing popularity among talent acquisition companies and divisions (Jha et al., 2025). ML systems can evaluate resumes based on their content and context, aligning them more precisely with the job description. This way, ML methodologies can determine the most relevant candidates for the job advertisements.

1.2 Purpose Statement and Research Questions

Recruiters are facing significant challenges in screening a growing number of resumes during the recruitment process due to time constraints. Moreover, manual HR screening is prone to unconscious biases that can affect hiring decisions and candidate diversity. According to Dhobale et al. (2025), recruiters may favour candidates from certain educational backgrounds and regions, contributing to less inclusive hiring practices. Research shows that resumes with ethnic-sounding names are 28% less likely to receive callbacks, even if the qualifications are identical (Dhobale et al., 2025). In addition to the restriction caused by human screening processes, traditional applicant tracking systems (ATS) based primarily on keyword-based algorithms pose further limitations by overlooking qualified candidates who fail to optimise their resumes with job-specific keywords. Therefore, the current study will focus on exploring different types of machine learning techniques to provide the most effective matching between candidate profiles and job advertisements. It will also provide insight into the various circumstances under which certain techniques outperform others. The goal is to provide findings in the field of machine learning so that automatic recruitment systems can be applied in practice by business organisations during their hiring process. The following research question is answered in this research work:

- **RQ:** *Which machine learning techniques (e.g., similarity-based methods, deep learning embeddings, natural language processing) provide the most effective*

matching between candidate profiles and job advertisements, and under what circumstances do certain techniques outperform others?

1.3 Topic Justification

Modern applicants need to follow strict practices when applying for job roles in the contemporary marketplace to be hired. Relevant keywords need to be present in their resumes to pass the initial screening, which is often conducted by traditional applicant tracking systems. These systems prescreen candidates solely based on the presence of specific industry keywords. Applicants struggle to create Applicant Tracking System-compliant resumes (ATS), which results in poor automated recruitment screening (Jha et al., 2025). It has been estimated that between 60% and 75% of the resumes never get through the ATS (Jha et al., 2025), and many recruiters do not read the CVs if they haven't passed the initial screening. Resumes that do not use the industry-relevant key terms often do not reach the hiring team, and they often shortlist highly skilled individuals. Traditional applicant tracking systems are cumbersome and unable to provide accurate results compared to AI-based systems. This inefficiency hinders organisations from hiring the best talent, potentially reducing team productivity and innovation (Jha et al., 2025).

Online recruitment nowadays plays a crucial role in connecting job seekers with potential employers. Digital platforms have tremendously increased the number of candidates from across the world. According to a report from LinkedIn, there were 20 million job listings on LinkedIn and 660 million users from over 200 countries and territories all over the world as of late November 2019 (Bian et al., 2020). Therefore, it has become essential for contemporary business organisations to automate job-resume matching with the use of effective techniques that would be able to pinpoint highly qualified individuals.

Machine learning techniques provide improved automation, allowing recruiters to spend minimal time on manual work and enhancing the speed of hiring decisions. Case studies also suggest that AI-driven recruitment tools can increase hiring accuracy and diversity when used correctly, improving transparency and fairness (Jha et al., 2025).

The current research focuses on exploring diverse machine learning models and comparing their effectiveness using different datasets. The results could contribute to the pool of existing data about job-resume matching techniques that companies can adopt during their hiring processes. This would contribute to the creation of enhanced applicant tracking systems that can attract top-tier candidates, which would inevitably boost the productivity and revenues of business organisations.

1.4 Scope and Limitations

The current study aims to build several Machine Learning (ML) models for matching candidate profiles with job advertisements and exploring their effectiveness against each other and two different datasets. It also aims to observe and provide answers regarding what circumstances cause certain techniques to outperform others.

Due to time and resource constraints, this research will only evaluate several machine learning techniques, leaving space for future research on other types of ML models. The effectiveness of the chosen models is tested with only two datasets due to time constraints. Further research could explore the performance of the models against other datasets. Moreover, these outcomes could be compared to the results produced by manual human selections to evaluate the efficiency between human and machine findings.

1.5 Knowledge Gap

There is a considerable amount of research in the field of data science regarding various job resume matching techniques. Each study explores a variation of a different methodology and models combined with a specific dataset. The current study adds to the pool of knowledge by contributing with another 11 distinct sets of techniques for resume job matching and provides an overview of the test results of their evaluation using a public Kaggle dataset and a private dataset provided by MindDig that has not been disclosed for public usage so far.

1.6 Ethical Considerations and Bias

Due to a non-disclosure privacy agreement with the company MindDig, we are unable to provide a detailed overview of the dataset text and examples of the generated resume-job pairs. Only a glimpse of the job fields and dataset columns is provided. In addition to this, we have also avoided using personally identifiable information in any of the datasets to protect the privacy of the individuals. The current study did not focus on avoiding gender bias specifically, but the results are considered not to be biased towards gender since neither of the datasets contains information about the gender of the participants. Even if the models are tested on other datasets, gender bias is unlikely to occur since all models only compare the job titles with a matching resume. Thus, the models are considered to be bias-free in terms of gender, ethnicity, nationality, etc. Therefore, job-resume matches are solely based on previous experience, education, and job title.

2. Review of the Literature

2.1 Search strategy

The literature review is conducted with scientific articles from the following IT & electronics databases:

1. Academic Search Premier
2. ACM Digital Library
3. ASTM Compass
4. IEEE Xplore
5. ScienceDirect Journals & Books
6. ScienceDirect Topics
7. Scopus
8. SPIE Digital Library

9. SpringerLink
10. Web of Science

The following keywords have been used during the search:

1. "matching profiles with job advertisements, machine learning"
2. "resume matching"
3. ("word embeddings" OR "deep learning embeddings" OR "vector representation")
4. AND ("job matching" OR "recruitment system" OR "profile-job fit")
5. ("similarity-based methods" OR "word2vec" OR "BERT" OR "TF-IDF" OR "deep learning")
6. AND ("job matching" OR "resume ranking" OR "profile-job fit")

2.2 Decision Tree-Based Models

To effectively optimize recruitment strategies and enhance the intelligence level of talent selection, Li & Yuan (2025) designed a recruitment strategy optimization method based on big data modelling using large enterprise recruitment data. The model utilised the LightGBM (Light Gradient Boosting Machine) decision tree algorithm, and the results showed improved matching prediction with an accuracy rate of 78.6%, precision of 75.4%, recall rate of 80.1%, and F1 score of 77.7% (Li & Yuan, 2025). The model was tested using data from a large enterprise over the years, including candidates' personal information, educational background, work experience, skill evaluation, interview performance, and final employment results (Li & Yuan, 2025). The dataset was split into technical and management positions during the testing phase.

2.3 Classical Machine Learning Models

Other authors focused on traditional ML models like Support Vector Machine (SVM), logistic regression, and k-Nearest Neighbors (KNN) to enhance job-resume matching. Surya et al. (2024) employed natural language processing (NLP) and machine learning (ML) techniques to evaluate applicants in real time. They used tokenization, stemming, lemmatization, TF-IDF, and string matching to identify keywords (Surya et al., 2024). Four ML models were applied to determine the most suitable role for each applicant - SVM, logistic regression, random forest, and k-nearest neighbours (Surya et al., 2024). Results showed that SVM and random forest achieved the highest accuracy for the training dataset, followed by logistic regression and KNN (Surya et al., 2024). The model was trained using a dataset with 963 rows and two columns: job category and a corresponding job resume.

Kamkar et al. (2024) developed another model that preprocesses textual data through tokenization, Named Entity Recognition (NER), and Part-Of-Speech (POS) tagging, utilizing spaCy's advanced linguistic annotations. After the preprocessing, the system utilized SVM for classification to match candidate profiles with job descriptions based on the extracted features (Kamkar et al., 2024). SVM was chosen for its effectiveness in handling high-dimensional data and capturing nonlinear relationships between textual features (Kamkar et al., 2024). The model was tested using a collection of anonymized resumes from a diverse range of industries

and job roles, and it demonstrated improved matching results compared to classical keyword matching algorithms (Kamkar et al., 2024).

In another study, Pang (2024) used a combined approach of Word2Vec for building embeddings and other machine learning methods (SVM, KNN, and cosine similarity) to create different classifiers. Pang (2024) initially collected resumes received by a Hong Kong company for data engineering positions over the past three years and labeled them as qualified or unqualified. The Word2Vec model was afterwards trained using the textual information of job descriptions of 5000 data engineers from various industries and companies (Pang, 2024). W2V performance was combined with different classification algorithms in the experimental data (Pang, 2024). W2V & SVM model showed relatively better results compared to the other two combinations - W2V & KNN and W2V & cosine similarity. Results also showed that Word2Vec is suitable for practical use in medium-sized enterprises that lack substantial hardware and human resources, where developing large models may not be feasible (Pang, 2024). This approach best fits situations where resources are limited. It can successfully achieve the goal of intelligent recruitment with constrained computational resources (Pang, 2024).

Another research performed by Maree & Shehada (2024) studied the effectiveness of Large Language Models (LLMs) (such as GPT-4, GPT-3, and LLAMA) and traditional models (such as Logistic Regression, Decision Tree, Naïve Bayes, and SVM). The authors used Kaggle's Updated Resume Dataset (Jillani SofTech, 2022), consisting of 962 resumes and multiple job categories in key industries such as finance, technology, and healthcare (Maree & Shehada, 2024). The overall outcomes of the study showed that traditional models such as Logistic Regression, Decision Trees, Naive Bayes, and SVM demonstrated robust performances when applied to structured data. On the other hand, LLMs such as GPT-3.5-turbo and LLAMA exhibited exceptional proficiency in interpreting complex, unstructured textual information (Maree & Shehada, 2024). However, LLMs exhibited certain challenges in concept-mismatching and ambiguity in comprehending domain-specific knowledge, proving that these types of models need extensive fine-tuning to mitigate this weakness (Maree & Shehada, 2024).

2.4 Deep Learning Models (CNNs, RNNs, BiLSTM, Attention)

Rezaeipourfarsangi & Milios (2023), Susan et al. (2024), and Zhang (2024) focused on studying the effectiveness of different types of neural architecture, including Convolutional Neural Network (CNN), Long Short-Term Memory (LSTM), Bidirectional Long Short-Term Memory (BiLSTM), and Word2Vec. Rezaeipourfarsangi and Milios (2023) created a system utilizing CNN to extract local features, while simultaneously employing LSTM to capture global features. The authors apply a multi-head attention layer that allows token representations to be influenced by the most relevant tokens within a document, providing a more fine-grained and context-aware representation (Rezaeipourfarsangi & Milios, 2023). The model is compared to standard methods, including Siamese CNN (S-CNNs), Siamese LSTM with Manhattan distance, and a BERT-based sentence transformer model using a dataset from a

Canadian recruitment company that provided 4,198 job descriptions and 268,549 resumes (Rezaeipourfarsangi & Milios, 2023). Findings showed that the system surpassed the performance of the other models in terms of various evaluation metrics such as F1 score, precision, recall, and accuracy (Rezaeipourfarsangi & Milios, 2023).

Susan et al. (2024) developed another system to extract semantic representations of only those keywords that occur more frequently in one class than in any other class in resumes. The sequence of unique elite keywords extracted from each resume was transformed into semantically meaningful GloVe and Word2Vec word embeddings, which are then fed to a BiLSTM classifier as sequential input (Susan et al., 2024). Using two different datasets – one from the site livecareer.com and one from Github (Jiechieu & Tsopze, 2020), the authors obtained an accuracy of 70.64% on the first one and 93.77% on the second one (Susan et al., 2024). The use of attention layers was found to degrade the performance in the case of Dataset-1, indicating that context information does not matter for unrelated job profiles (Susan et al., 2024). The attention mechanism was found to boost the results for Dataset-2, underlining the significance of context in the resume documents belonging to this dataset, in which the job profiles are highly interrelated (Susan et al., 2024). The proposed approach outperforms the state of the art by a significant margin, proving that it is better than the individual bag-of-words model and the word-embedding-based models popularly used in literature for the classification of text documents (Susan et al., 2024).

In a related study, Zhang (2024) proposed a system that utilizes natural language processing based on deep learning. It uses a two-way long-term memory network and attention mechanism to achieve an accurate understanding of text content. The author combined it with an OCR system based on a convolutional neural network (CNN) to improve the recognition accuracy of non-standard text (Zhang, 2024). They used data from public recruitment platforms to test the system, and the results showed that the model significantly improved the efficiency of traditional manual screening (Zhang, 2024). The simulation experiment revealed that after processing 2,000 resumes in various formats, the system achieved 98% text recognition accuracy, 95% information extraction accuracy, and 93% job matching accuracy, verifying the practicality and reliability of this method (Zhang, 2024).

Research conducted by Ni (2022) combined deep neural network technology with digital Human Resource Management (HRM) knowledge to research human-job matching. The job intelligent recommendation algorithm adopted a bidirectional long and short-term memory neural network (Bi-LSTM) and the word-level human post-matching neural network APJFNN built by the attention mechanism (Ni, 2022). By embedding the text representation of job demand information into the representation vector of public space, a joint embedded convolutional neural network (JE-CNN) for post matching analysis is designed and implemented (Ni, 2022). The model was tested using the data generated by a corporate system when candidates applied for a job. Results showed that the model performs differently for different categories of jobs (Ni, 2022). The AUC performance of this model is relatively good for technical positions, while for other types and civil service, the performance is relatively weak (Ni, 2022).

Expanding on earlier work, Drigas et al. (2004) created an expert system for the evaluation of the unemployed for certain offered posts. The system uses a Sugeno-type Neuro-Fuzzy inference system, which combines the strengths of neural networks and fuzzy logic to model complex, non-linear systems (Drigas et al., 2004). The model was trained on a dataset of currently unemployed citizens (rejected or approved at several posts and belonging to the same social class) provided by the Greek General Secretariat of Social Training (Drigas et al., 2004). The percentage of applicants that were hired was assigned weights manually selected with equal importance ($w_i = 0.5$) instead of weights learned by training (Drigas et al., 2004). The fields of age and previous employment were found to be very strongly weighted by the system (Drigas et al., 2004). Results showed that the performance of the system for every specialty is different, but learning weights lead to an overall average increase of 10% in successful job matching (Drigas et al., 2004).

2.5 Transformer-Based and LLM Models

Bevara et al. (2025) presented a model named Resume2Vec to create embeddings for resumes and job descriptions using cosine similarity. The model utilizes transformer-based deep learning models, including encoders (BERT, RoBERTa, and DistilBERT) and decoders (GPT, Gemini, and Llama). Using 15,000 anonymized resumes, the system was evaluated through a quantitative analysis via embeddings and a qualitative human assessment across several professional fields (Bevara et al., 2025). The results showed that Resume2Vec outperformed conventional Applicant Tracking Systems (ATS), achieving enhancements of up to 15.85% in Normalized Discounted Cumulative Gain (nDCG) and 15.94% in Ranked Biased Overlap (RBO) scores, especially within the mechanical engineering and health and fitness domains (Bevara et al., 2025). ATS exhibited slightly superior nDCG scores in operations management and software testing, but Resume2Vec displayed a more robust alignment with human preferences across the majority of domains (Bevara et al., 2025).

Another relevant contribution comes from Dilli Ganesh et al. (2025), who created a system that combines NLP and neural networks. They used a Bidirectional Encoder Representations from Transformers (BERT) model for resume parsing and applied information extraction (Dilli Ganesh et al., 2025). A Siamese neural network architecture comprising of two distinct networks was used to learn features of resumes relative to those of a job description (Dilli Ganesh et al., 2025). The networks were trained in such a way that distances between embeddings of matching resumes and the job descriptions were minimized, and at the same time, the distances among the non-matching ones were maximized (Dilli Ganesh et al., 2025). The model was trained using a combination of datasets, including the Kaggle Resume Dataset. Precision, recall, F1-score, and accuracy were applied to evaluate the model (Dilli Ganesh et al., 2025). The results showed an overall high satisfaction score of HR practitioners who evaluated the system. It was estimated that the model makes fair and ethical decisions regarding talent acquisition (Dilli Ganesh et al., 2025). The high precision and recalls, together with the high F1-Scores, suggested that the model is very good in identifying which candidates are suitable for certain positions (Dilli Ganesh et al., 2025).

A separate study by Dhobale et al. (2025) developed an AI resume ranking system called ResuMatcher, which integrates LLM models such as BERT and Gemini – advanced systems, for contextual analysis of resumes and job descriptions to compute semantic matching scores (Dhobale et al.,2025). The system consists of two modules – the Job Seeker Module and the Recruiter Module (Dhobale et al.,2025). The Job Seeker Module enables candidates to register, upload resumes, and receive detailed feedback on their applications (Dhobale et al.,2025). It also provides match scores to help users understand their compatibility with job descriptions (Dhobale et al.,2025). The Recruiter Module automatically ranks candidates by analyzing the semantic relevance of their resumes against the job criteria and allows recruiters to post job requirements specifying the desired skills, qualifications, and experience (Dhobale et al.,2025). The system was tested with real-world resumes provided by recruiters in diverse industries, including healthcare, IT, and finance (Dhobale et al.,2025). The results were compared against the outcomes of traditional keyword-based recruitment systems (Dhobale et al.,2025). They showed that 80% of recruiters found the ranked resumes highly relevant to job requirements, and the system reduced the average hiring process time by 30% (Dhobale et al.,2025). 85% of users reported that the match percentages aligned with their expectations (Dhobale et al.,2025). The ResuMatcher outperformed traditional keyword-based ATS systems, showing an average improvement of 15% in F1-score and 10% in accuracy (Dhobale et al.,2025).

Complementing the above studies, Rosenberger et al. (2025) introduced an advanced support tool for career counselors and job seekers based on CareerBERT, leveraging the BERT (Bidirectional Encoder Representations from Transformers) architecture. The model was trained using data from the European Skills Competences Occupations (ESCO) taxonomy and European Employment Services (EURES) job advertisements (Rosenberger et al., 2025). It was evaluated using EURES job advertisements and human-grounded evaluations. Results showed that CareerBERT outperforms both traditional and state-of-the-art embedding approaches while showing robust effectiveness in human expert evaluations (Rosenberger et al., 2025).

Similarly, Dharmendra et al. (2025) used natural language techniques to identify the main features in a resume. The authors introduced an improved ranking framework utilizing BERT embeddings to identify and evaluate relevant features. Transformers and especially BERT were used to improve accuracy and context awareness by capturing semantic similarities between the words on the resume and job postings (Dharmendra et al., 2025).

The performance of the system was evaluated through precision, recall, F1-score, and Mean Reciprocal Rank (MRR) metrics (Dharmendra et al., 2025). Results were then compared against several other resume matching techniques, such as TF-IDF-based ones, LSTMs, and Random Forest classifiers, proving the BERT classifier as the most accurate model (Dharmendra et al., 2025). The data showed that the word analysis method of TF-IDF leads to incorrect results because it does not understand contextual word meaning, as the method cannot recognize synonyms or interpret words semantically due to technical limitations (Dharmendra et al., 2025). The resume screening system is more effective after adopting BERT because it detects word dependencies and semantic similarities, which outperform TF-

IDF functionality (Dharmendra et al., 2025). The TF-IDF and BERT-based models were compared using a 3000-record Dataset (Dharmendra et al., 2025).

The system was also tested in a real-world environment and evaluated by HR professionals from multiple organizations (Dharmendra et al., 2025). It achieved a 20% reduction of time spent on resume screening, an 85% satisfaction level regarding the accuracy of candidate ranking, and a 92% approval of the usefulness of keyword highlighting (Dharmendra et al., 2025).

In a related study, Alonso et al. (2025) created an automated job-resume matching system that they claim is bias-free, as it did not rely on data from previous applicants. Alonso et al. (2025) created a model that fixed the cold start issue, which arises in collaborative filtering when the system faces difficulties in making appropriate recommendations due to a lack of sufficient user data.

The authors employed a Transformer-based sentence embedding approach using the SentenceTransformers framework with the embedded all-mpnet-base-v2 model (Alonso et al., 2025). They used 105 resumes from 21 different categories to compare them against a Kaggle Resume Dataset (Dutta, G., 2021) while testing one out of two scenarios (Alonso et al., 2025). The second scenario was tested using 1000 job postings from the CareerBuilder job dataset (Alonso et al., 2025).

The results were evaluated through a scoring mechanism with five different values from 1 to 5 (1 being completely different from the original one and 5 being perfectly matching the original one) (Alonso et al., 2025). The authors created two different prediction systems – baseline 1 and 2, which were manually evaluated by human annotators, and the two baseline approaches were compared against the ML model (Alonso et al., 2025). Results showed that the AI system generated better outcomes than the manual human evaluations, scoring 3.8 for the first scenario compared to 3.4 and 3.0 for the two baselines for annotation one (Alonso et al., 2025). The custom approach also scored 4.17 for annotation one in the second scenario compared to 3.8 and 2.03 for the baselines (Alonso et al., 2025).

An additional investigation by Gu et al. (2025) tackled the cold start problem by leveraging resume data as a source of information to generate accurate results for new users even in the absence of historical interaction data (Gu et al., 2025). The cold start problem refers to the challenge a system faces when it has insufficient data about a new user or new item to provide accurate predictions or recommendations. Gu et al. (2025) also addressed the long-tail problem by utilizing external knowledge sources such as a knowledge graph which builds entities such as job roles, skills, industries, degrees, certifications, etc. and connects them through semantic relationships like: “requires skill”, “is similar to”, “is part of industry”, “is related to degree” (Gu et al., 2025). The long tail problem refers to a situation where a dataset exhibits a long-tailed distribution when there's a small number of high-frequency "head" classes and a very large number of low-frequency "tail" classes. This causes models to struggle with effective learning of the scarce examples in the tail classes, which results in poor performance.

The authors proposed an Online Job Recommendation model via Resume Fusion (OJRRF) aimed at making accurate and efficient online job recommendations (Gu et al., 2025). It consists of two submodules: a knowledge-aware offline recommendation model and a content-based online recommendation model (Gu et al., 2025). The knowledge-aware model first leverages the BERT model to obtain user resume vectors and then integrates them with the Knowledge Graphs (KG) attention model to enhance the user representation (Gu et al., 2025). The content-based online recommendation module applies cosine similarity calculations to determine the similarity between pairs of jobs (Gu et al., 2025). The system was tested using a Chinese online recruiting platform named “JiuYeJie”. Its performance was compared against classic and state-of-the-art methods for recommendation, such as KG-based recommendation models, random forest, RNN, and others (Gu et al., 2025).

Results showed the superiority of the proposed model, which improved the accuracy, reliability, and timeliness of the recommendation results (Gu et al., 2025).

2.6 Hybrid & Multimodal Systems

Other authors have focused their studies on developing systems that combine image, rule-based, or mixed ML/NLP methods. Yazıcı et al. (2024) created a resume-ranking web application that allows HR professionals to upload resumes seamlessly, define job descriptions, and view ranked results. Using a custom dataset for object detection training, they fine-tuned a YOLOv9 model for segment detection on resumes, EasyOCR for text recognition, mBERT for text classification, and GLiNER for named entity recognition with regular expressions (Yazıcı et al., 2024). Results showed that the YOLOv9 model achieved the highest performance when compared against two other models (DETR and Detectron2) with a score of 0.84 mAP (Yazıcı et al., 2024).

In another study, Ramyar et al. (2024) developed a system that generates tailored interview questions specific to each candidate's profile and sends emails to those people whose resumes do not match the job description, stating what is lacking in their resume when compared to the job description. This reduced the HR's work for preparing interview questions for each person and provided personalized feedback to non-selected candidates (Ramyar et al., 2024). The authors employed transformer models like BERT, RoBERTa, and GPT-3 to accurately categorize and rank resumes. A hybrid Named Entity Recognition (NER) approach combined a rule-based approach and machine learning methods in order to extract key information (Ramyar et al., 2024). Interview questions were generated using NLP techniques (Ramyar et al., 2024). The system was trained on a Kaggle dataset, and the results showed that the system achieved 98% accuracy (Ramyar et al., 2024).

Similarly, Agarwal et al. (2025) developed a resume reviewing system for students used for job matching and placement prediction tools to assist them in their employment search called OpportuNest (Agarwal et al., 2025). The model was tested on historical records of student enrollment and applications from partner companies (Agarwal et al., 2025). OpportuNest comprises a combination of AI and ML approaches. OpportuNest utilizes an LSTM (Long Short-Term Memory) Neural Network algorithm to forecast trends such as student enrollment and placement rates (Agarwal et al., 2025). Logistic Regression is used to predict the success

rate of job placements and evaluate employability by analyzing students' academic records, test scores, and additional data (Agarwal et al., 2025). Bidirectional Encoder Representations from Transformers (BERT) is also applied to enhance how well resumes match job specifications (Agarwal et al., 2025). Results showed that the model provides improved campus placements and recruitment, making the placement procedure faster and more trustworthy (Agarwal et al., 2025).

Expanding on earlier work, Bian et al. (2020) proposed a multi-view co-teaching network – a combined approach of a text-based and relation-based matching model, which can learn from sparse, noisy interaction data for job-resume matching. The text-based matching model adopts a hierarchical self-attention text encoder for capturing text semantics, which uses a BERT-based encoder to represent sentences. Afterwards, a Transformer-based encoder is applied to represent the overall document based on learned sentence embeddings (Bian et al., 2020). The relation-based model embeds a graph neural network that creates a relation graph, regarding the jobs and resumes as nodes and their underlying correlations as links (Bian et al., 2020). This job-resume relation graph captures implicit correlations and then develops a matching model using relational graph neural networks (Bian et al., 2020).

The proposed model was evaluated on a real-world dataset provided by a popular Chinese online recruiting platform named “BOSS Zhipin” (the BOSS Recruiting) (Bian et al., 2020). The results were compared to several other representative methods, among which are DSSM, NFM, BPJFNN, PJFNN, APJFNN, JRMPM, and UBD (Bian et al., 2020). The models were evaluated using the evaluation metrics AUC, accuracy, precision, recall, and F1 score (Bian et al., 2020).

The outcomes showed that the proposed multi-view co-teaching network achieved the best performance on all the metrics across different datasets and improved the best baseline on average by 3.1%, 2.9% and 3.5% on the datasets of Technology, Sales, and Design, respectively (Bian et al., 2020). The co-teaching mechanism in Bian et al. (2020) model could identify more informative and reliable samples for learning the parameters, and it was more resistant to the negative impact brought by noisy data. Thus, the multi-view co-teaching network performed better than the other comparison methods (Bian et al., 2020).

Complementing the above, Li & Yuan (2024) designed a recruitment strategy optimization method based on the LightGBM (Light Gradient Boosting Machine) decision tree algorithm to effectively optimize recruitment strategies and enhance the intelligence level of talent selection based on big data modeling using large enterprise recruitment data. Results showed improved matching prediction with an accuracy rate of 78.6%, a precision of 75.4%, a recall rate of 80.1%, and an F1 score of 77.7% (Li & Yuan, 2024).

2.7 Embedding and NLP-Based Models

Vanetik & Kogan (2023) and Biol & Abalorio (2024) focused on developing systems that utilize TF-IDF, Word2Vec, sentence embeddings, and NER, but not neural networks directly. Vanetik and Kogan (2023) proposed a method for ranking job vacancies using a combination of sentence embeddings, keywords, and named entity recognition (NER) to achieve state-of-

the-art performance in resume matching (Vanetik & Kogan, 2023). The method called VectorMatching(VM) is based on vector similarities and uses BERT-based extractive summarization with a pre-trained English model bert-base-uncased to obtain summaries for vacancies and resumes (Vanetik & Kogan, 2023). The system was trained on 500 million jobs titled “software developer” extracted over the past 3 years from USA job websites (Vanetik & Kogan, 2023). It also utilizes real resumes from the Telegram group “HighTech Israel Jobs” (Vanetik & Kogan, 2023). The model was evaluated using both human and automatic annotations and compared against two baselines—OKAPIBM25 and BERT (Vanetik & Kogan, 2023). Results showed that using full-text resumes along with n-gram-based text representation generated the best performance, while TF-IDF-based representations yielded the lowest performance (Vanetik & Kogan, 2023).

The second research conducted by Biol & Abalorio (2024) focused on creating a web-based resume scorer, using Named Entity Recognition (NER) to automate the evaluation of resumes in the hiring process. A dataset of 1,014 resumes annotated by Roman Shilpakar was utilized by fine-tuning the RoBERTa NER model using Spacy transformers (Biol & Abalorio, 2024). The results demonstrated improvements in model performance over training epochs, with increased precision, recall, and F1 scores (Biol & Abalorio, 2024).

3. Methodology

3.1 Datasets

The current study works with two different datasets - one provided by the MindDig company and one downloaded from the Kaggle website. MindDig is a Swedish digital recruitment and talent attraction platform based in Luleå, Sweden, that connects skilled individuals with companies. The MindDig company focuses on developing tools for AI matching of candidates with prospective employers using a vast database of CVs and Job advertisements. For the purpose of the current study, the company provided a dataset of more than 6000 resumes and more than 1600 Job postings from various fields, which are visible in Appendix A.

The MindDig resume dataset contains 9 different columns - “about” (short description of the candidate), “tools”, “skills”, “location”, “languages”, “education”, “certificate”, and “driver's licence”. Some of the files also contain a 10th column labeled “linkedin”. The MindDig Job dataset contains another 6 columns - “title_en” (the job title), “description_end” (the description of the job position), “work_model” (office, hybrid, or remote), “employment_type” (part or full time), “drivers_license,” and “location”.

The second dataset was acquired from the Kaggle website, and it is available in the references below (Jillani SofTech, 2022). It contains two columns - one with resume descriptions and one with category labels (Fig.1) from more than 20 different domains (Appendix A). The dataset consists of nearly 1000 rows of categories matched to a corresponding resume. Unlike the MindDig dataset, the Kaggle dataset is humanly annotated, meaning that each resume is already linked to the correct job category.

	Category	Resume
0	Data Science	Skills * Programming Languages: Python (pandas...
1	Data Science	Education Details \r\nMay 2013 to May 2017 B.E...
2	Data Science	Areas of Interest Deep Learning, Control Syste...
3	Data Science	Skills â€ R â€ Python â€ SAP HANA â€ Table...
4	Data Science	Education Details \r\n MCA YMCAUST, Faridab...

Fig 1. Kaggle dataset snippet before data preprocessing

3.2 Data preprocessing

Before the models are built, NLP preprocessing is applied to both MindDig and Kaggle datasets to prepare them for resume job matching. Text cleaning is performed by removing HTML tags, email links, and special characters (only letters, numbers, and whitespaces are kept; punctuations like . , ! ? are removed). Lowercasing, tokenization, and lemmatization are applied afterwards. Common English stop words such as “*the, and, is, of, to*” are also removed. Additional preprocessing is applied on the Kaggle dataset, as it contains non-readable characters that were likely bullet points before the text was converted and uploaded to Kaggle. All non-ASCII characters, such as â, €, ™, etc., that appeared in the text were removed. If a value is not a string (e.g., NaN), it is preprocessed as an empty string “”. This is done, so the model would compare the important content instead of noise.

After preprocessing, duplicates are removed from the job description (“description_en” column) and resume description columns (“about” column) of the MindDig datasets and the resume description (“resume” column) of the Kaggle dataset. The columns “about”, “tools”, “skills”, “educations”, “experiences”, and “certifications” from MindDig’s resume dataset are united into one column labeled as “resume_summary”. Only the columns “title_en” indicating the job category (e.g., HR, Data scientist, etc.) from the job ads in the MindDig dataset are used for comparison against the resumes.

3.3 Models

Three different families of ML models were used to match job descriptions to the most relevant resumes by measuring semantic similarity between their text embeddings.

3.3.1 Semantic embedding model + cosine similarities

The first model uses a pretrained semantic embedding model called “text-embedding-ada-002”, developed by OpenAI. This is the second-generation embedding model in the text-embedding-ada series. It is a machine learning model that transforms natural language into a fixed-length numerical vector (an embedding) (OpenAI, 2022a). For each input, it produces a 1536-dimensional vector (OpenAI, 2022a). The embeddings capture the meaning of text, so similar words or sequences of words are positioned close together in the vector space (OpenAI, 2022a).

This model is trained for general-purpose embeddings across dozens of tasks (semantic search, clustering, classification, recommendations, and beyond) (OpenAI, 2022a). OpenAI unifies five embedding models—including text search, text similarity, and code search—into this single model (OpenAI, 2022a). Ada v2 shows strong performance compared to prior models when matching semantic similarity between sentences, and it is particularly powerful for resume job-matching (OpenAI, 2022a).

Instead of relying on rule-based keyword matching, this model captures semantic similarities in text through semantic embeddings, allowing it to understand the meaning of synonyms. For example, “Senior Java Developer” and “Backend Engineer with Java experience” would be positioned in vectors close to each other, whereas “Nurse” and “Electrical Engineer” would be further apart in the vector space.

The “text-embedding-ada-002” model is based on the Transformer architecture (similar to GPT models) and uses an attention mechanism to capture relationships between words within a given context. It is a pre-trained model on large-scale text corpora to capture semantic similarities. The model is a cloud-based hosted service, and it is available through API calls (via `openai.Embedding.create()` or method (legacy) or via the new client SDK `client.embeddings.create()`) (OpenAI, 2022a). The API calls are not free, but relatively cheap per token.

The text-embedding-ada-002 model demonstrated superior performance in semantic search tasks over previous models when tested by OpenAI (OpenAI, 2022a). It was evaluated on the SentEval (STS 2012–2016) dataset (OpenAI, 2022a). The results (Fig.2) indicate that text-embedding-ada-002 outperforms all previous embedding models in semantic search tasks (OpenAI, 2022a).

Model	Performance
text-embedding-ada-002	81.5
text-similarity-davinci-001	80.3
text-similarity-curie-001	80.1
text-similarity-babbage-001	80.1
text-similarity-ada-001	79.8

Dataset: [SentEval](#) (STS 2012–2016)

Fig.2 Sentence similarity comparisons of OpenAI Semantic embedding model (OpenAI, 2022a).

The text-embedding-ada-002 merges five separate OpenAI models (text-similarity, text-search-query, text-search-doc, code-search-text, and code-search-code) into a single new model (OpenAI, 2022a). The context length of the new model is increased by a factor of four, from 2048 to 8192, making it more convenient to work with long documents (OpenAI, 2022a). The new embeddings have only 1536 dimensions, one-eighth the size of davinci-001 embeddings, making the new embeddings more cost-effective in working with vector

databases (OpenAI, 2022a). The price of new embedding models was reduced by 90% compared to old models of the same size. As a result, the new model achieves better or similar performances as the old Davinci models at a 99.8% lower price (OpenAI, 2022a).

The “text-embedding-ada-002” model has certain limitations, and it does not outperform “text-similarity-davinci-001” on text classification tasks (OpenAI, 2022a). However, when it comes to sentence similarity tasks such as job resume matching, the Ada 2 model has proven to be the most effective out of all 5 models evaluated by OpenAI (OpenAI, 2022a).

OpenAI “text-embedding-ada-002” was chosen in the current research because it produces embedding quickly and at a relatively lower cost. It also works effectively when no labeled training data is available, and it is compatible with unsupervised learning. This makes it suitable for our MindDig datasets, which do not contain pre-matched job-resume pairs (OpenAI, 2022a). The Ada 2 model also does not require additional fine-tuning and performs well across different domains, making it a good fit for cross-industry datasets (OpenAI, 2022a). After each resume and each job is transformed into a separate 1536-dimensional embedding using the OpenAI “text-embedding-ada-002”, cosine similarity was applied to translate the embedding vectors into a practical ranking of resumes for each job posting. This ensures that resume job matching is based on semantic closeness, instead of keyword overlap.

3.3.2 SentenceTransformer + cosine similarities

The second family of ML techniques comprises of 9 different types of embedding models. Each of these sentence transformers is combined with cosine similarity techniques. A Sentence Transformer is a type of neural network model that transforms text into dense vector embeddings that capture semantic meanings. Sentence transformers produce fixed-size vectors for entire sentences, making them suitable for semantic similarity tasks such as resume job matching. This makes them more suitable compared to the standard transformer models (such as BERT or T5), which create token-level embeddings that output each word or subword individually, failing to fully capture the sentence-level meaning (Reimers & Gurevych, 2019).

Nine different sentence transformer models were used in the current study in combination with cosine similarities. All of these models were developed and maintained by the UKP Lab (Ubiquitous Knowledge Processing Lab) at the Technical University of Darmstadt, often in collaboration with Hugging Face (Reimers & Gurevych, 2019). The foundational concept of this model originates from a 2019 research paper - “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks” by Nils Reimers and Iryna Gurevych. It was later developed in the Sentence Transformers library and published at the official documentation site for the Sentence Transformers library [sbnet.net](https://www.sbert.net) (UKP Lab & Hugging Face, 2025).

3.3.2.1 all-MiniLM-L6-v2

The first sentence transformer model, which was used in this study, “all-MiniLM-L6-v2” maps sentences and short paragraphs into 384-dimensional dense vectors and was trained using a 1 billion sentence pairs corpus. The model was fine-tuned via contrastive learning objectives, and it is given pairs of sentences (positive pairs: semantically similar; negative pairs:

unrelated) to learn to maximize similarity for positive pairs and minimize similarity for negative pairs in the embedding space (Sentence Transformers, 2025a). It is suitable for semantic search and clustering (Sentence Transformers, 2025a).

3.3.2.2 all-mpnet-base-v2

The second model used in the current study – “all-mpnet-base-v2” is another sentence transformer model which uses the pretrained microsoft/mpnet-base model and maps sentences and paragraphs into a 768-dimensional dense vector space. As the previous model, this one was also fine-tuned using a contrastive learning objective on a dataset of over 1 billion sentence pairs, making it suitable for semantic search, clustering, and sentence similarity (Sentence Transformers, 2025b).

3.3.2.3 all-MiniLM-L12-v2

The “all-MiniLM-L12-v2” is another lightweight embedding model from the same Sentence Transformers family, which also transforms sentences and short paragraphs into 384-dimensional dense vectors and is optimized for semantic search and clustering (Sentence Transformers, 2025c). It is built upon the Microsoft MiniLM-L12-H384-uncased architecture and was also fine-tuned via contrastive learning using a 1 billion sentence pair corpus (Sentence Transformers, 2025c). According to Pinecone’s official documentation, all-MiniLM-L12-v2 is about 5 times faster than the all-mpnet-base-v2 model while still offering good quality (Pinecone, 2025).

3.3.2.4 multi-qa-MiniLM-L6-cos-v1

The next model “multi-qa-MiniLM-L6-cos-v1” builds upon the MiniLM architecture, which is a lightweight 6-layer transformer, and it also produces dense embeddings of 384 dimensions (Sentence Transformers, 2025d). The model is optimized for question-answer retrieval, and it is fine-tuned on multiple QA datasets using cosine similarity loss (Sentence Transformers, 2025d). It has been trained on 215M (question, answer) pairs from diverse sources, and it is suitable for semantic search, FAQ matching, and QA retrieval, where queries and documents are short texts (Sentence Transformers, 2025d).

3.3.2.5 multi-qa-distilbert-cos-v1

“Multi-qa-distilbert-cos-v1” is another transformer-based sentence embedding model based on DistilBERT - a distilled version of BERT that reduces the number of parameters while maintaining high performance (Sentence Transformers, 2025e). The model is particularly effective for retrieving answers corresponding to a given query, as it was fine-tuned on multiple question-answering datasets using a cosine similarity (Sentence Transformers, 2025e). The model generates 768-dimensional embeddings, providing a richer semantic representation compared to smaller models (Sentence Transformers, 2025e). It is optimized for question-answer retrieval, FAQ systems, and any scenario where matching semantically similar text pairs is required (Sentence Transformers, 2025e). The model is suitable for large-scale retrieval systems and balances accuracy with efficiency (Sentence Transformers, 2025e).

3.3.2.6 paraphrase-mpnet-base-v2

The next model used in this study - “paraphrase-mpnet-base-v2”, builds upon the MPNet model, which integrates the advantages of BERT and XLNet (Sentence Transformers, 2025f). It creates 768-dimensional dense vectors and was fine-tuned for paraphrase identification tasks (Sentence Transformers, 2025f). It is suitable for semantic textual similarity, paraphrase mining, and clustering of semantically similar sentences (Sentence Transformers, 2025f).

3.3.2.7 sentence-t5-base

Another model developed by the Sentence Transformer team, “sentence-t5-base”, also creates 768-dimensional embeddings and is based on the T5 (Text-to-Text Transfer Transformer) model, which treats all NLP tasks as text generation problems (Sentence Transformers, 2025g). It is suitable for semantic search, clustering, and other tasks requiring sentence-level embeddings (Sentence Transformers, 2025g). While it performs well for sentence similarity tasks, it may not be as effective for semantic search tasks (Sentence Transformers, 2025g).

3.3.2.8 all-distilroberta-v1

The “all-distilroberta-v1” is based on the DistilRoBERTa model, which is a distilled version of RoBERTa (Sentence Transformers, 2025h). The model is optimized for semantic search, clustering, and sentence similarity and generates 768-dimensional embeddings (Sentence Transformers, 2025h). It is fine-tuned on a large-scale dataset of 1 billion sentence pairs, including data from sources like Reddit, S2ORC, WikiAnswers, and PAQ (Sentence Transformers, 2025h).

3.3.2.9 gtr-t5-base

The last model from the same Sentence Transformers family “gtr-t5-base” uses the encoder of T5-base, a transformer-based architecture designed for text-to-text tasks. It produces 768-dimensional vectors (Sentence Transformers, 2025i). It is fine-tuned with supervised learning on large-scale semantic textual similarity datasets for general-purpose text embeddings, and it is suitable for semantic search, question answering, or recommendation systems (Sentence Transformers, 2025i). The model weights are stored in FP16, which reduces memory usage while maintaining high performance (Sentence Transformers, 2025i).

Model	Dimensions	Base Model	Optimized For
all-MiniLM-L6-v2	384	Microsoft MiniLM-L6	General-purpose semantic similarity, clustering, retrieval
all-MiniLM-L12-v2	384	Microsoft MiniLM-L12	General-purpose semantic similarity, clustering, retrieval
multi-qa-MiniLM-L6-cos-v1	384	Microsoft MiniLM-L6	Question-answer retrieval (dense passage retrieval)
multi-qa-distilbert-cos-v1	768	DistilBERT	Question-answer retrieval
all-mpnet-base-v2	768	Microsoft MPNet-base	General-purpose semantic similarity, clustering, retrieval
all-distilroberta-v1	768	DistilRoBERTa-base	General-purpose semantic similarity, clustering, retrieval
paraphrase-mpnet-base-v2	768	Microsoft MPNet-base	Paraphrase identification, similarity, clustering
sentence-t5-base	768	Google T5-base (seq2seq)	Universal sentence embeddings (NLI, STS, retrieval, clustering)
gtr-t5-base	768	Google T5-base	Dense retrieval, semantic search, QA

Fig.3 Summary table of the Sentence Transformers model used in the current study.

3.3.2.10 Cosine similarity

After encoding the resumes and jobs into dense vector embeddings using the SentenceTransformer models, cosine similarity was applied to evaluate how similar each resume is to the different job postings. The similarity is measured through the angle between vectors:

If the angle = $0^\circ \rightarrow$ cosine similarity = 1 (perfectly similar)

If the angle = $90^\circ \rightarrow$ cosine similarity = 0 (no similarity)

If the angle = $180^\circ \rightarrow$ cosine similarity = -1 (opposite direction)

3.3.3 TF-IDF + cosine similarities

Luhn (1958) suggested “that the frequency of word occurrence in an article furnishes a useful measurement of word significance” and his approach became known as term frequency weighting (Sanderson & Croft, 2012). Luhn’s term frequency (TF) weights were later complemented by Spärck Jones’s (1972) work on word occurrence across the documents of

a collection (Sanderson & Croft, 2012). Her paper on inverse document frequency (IDF) introduced the idea that the frequency of occurrence of a word in a document collection was inversely proportional to its significance in retrieval, meaning that less common words tended to refer to more specific concepts, which were more important in retrieval (Sanderson & Croft, 2012). The idea of combining these two weights (TF-IDF) was quickly adopted by Salton and Yang (1973). Later on in 1975, Salton, Wong, and Yang embedded TF-IDF in a Vector Space Model, showing improved retrieval performance (Salton, Wong, & Yang, 1975).

The third model in this research paper converts the resumes and job summaries into a numerical vector using TF-IDF. Then it computes the cosine similarity between job vectors and resume vectors. A higher similarity score indicates a stronger textual match between the job and the resume. Before vectorising the resumes and job summaries, common English stop words such as “the”, “and”, “is” are removed since they don’t contain important information and do not play a role in distinguishing between documents (`stop_words='english'`). Only the 10,000 most frequent terms across training data are kept by the `TfidfVectorizer`, so that rare/noisy words can get cut out (`max_features=10000`). Single words (unigrams) are used (`ngram_range=(1,1)`), as expanding to bigrams and trigrams resulted in decreased performance and a lower overall average similarity score. The `min_df=2` setting has been applied to ignore words that appear in fewer than 2 documents, since words that are too rare are more likely not to be useful. The `norm='l2'` setting has also been applied to ensure that each vector has unit length, which makes cosine similarity purely about direction and not about absolute length since we are measuring semantic similarity through angle comparison (direction only), independent of magnitude (document length).

The TF-IDF maps the words to a numerical index and computes the IDF value for each word, showing in how many documents that word appears. Common words across all resumes/jobs are assigned low IDF, and rarer words - high IDF. The similarity of job and resume summaries is then compared using cosine similarity, ranging from -1 to 1 (1 = identical vectors, perfectly similar; 0 = no overlap, no similarity, unrelated; -1 = perfectly opposite, completely dissimilar).

3.3.4 Why the above models were chosen for this research

The sequence-to-sequence transformer model in this research - “sentence-t5-base”, and the encoder-only embedding model - “text-embedding-ada-002”, were chosen as baseline models as they are trained on sentence similarity and semantic tasks over a large corpora. This makes them highly effective in recognizing semantically similar text, and they are also particularly powerful for resume job-matching. They work consistently well across different datasets and conditions, and they are stable, reliable, and less sensitive to noise. Both models demonstrate superior performance over the rest.

The rest of the transformer models were chosen because they produce sentence-level embeddings, making them suitable for semantic similarity tasks. The TF-IDF was chosen to show the effectiveness of modern embedding models over traditional keyword matching algorithms.

3.4 Train/Test/Validation Splits

The data is split into 80% training, 10% test, and 10% validation sets for all models.

Dataset train/test/validation split for TF-IDF

The combined resumes and jobs are fed into the TF-IDF vectorizer. Vocabulary and IDF weights are learned only from training data. The validation split is not utilized in this model, but can be used for future improvements, and it can be further adapted to tweak thresholds, hyperparameters (e.g., min_df, max_features), etc. Resumes and jobs from the test split are transformed into vectors, and cosine similarity is computed, so the metrics are calculated on unseen data from the test split.

Dataset train/test/validation split for the Semantic embedding model

The OpenAI comes with pre-trained embeddings and does not require model training. Therefore, no embeddings are computed for training and validation sets, and no model weight update is performed. The similarity scores and evaluation metrics are computed only on the test set. However, the training and validation can still be used to improve the model. The validation split can be utilized to tune hyperparameters, such as the similarity threshold to compute cosine similarity between validation resumes and jobs, and then test multiple thresholds. This would prevent “peeking” at the test set and help select the optimal threshold. The training set can be used to build a library of embeddings. The embeddings from the training and validation sets can be compared with the training job embeddings to find the most similar ones. This can potentially improve performance as the model will have more examples from the embedding library to compare against.

Dataset train/test/validation split for Sentence Transformers

The training and validation sets are also not used in the transformer model, and the evaluation is done on the test set, but they can be used to fine-tune the model and tune thresholds in the future.

3.5 Evaluation runs

The resumes are compared to jobs based on cosine similarity of embeddings. The evaluation is run multiple times with a random seed that controls how the resumes and jobs are shuffled and split, so that the evaluation can be tested under different random conditions. Running the evaluation with different seeds simulates different dataset splits. This checks if the model performs consistently. 5 independent evaluations of resume-job matching are run per value. Metrics are computed across multiple seeds to make more reliable estimations of how well

resumes match jobs, and the average values are reported. Since the tested pairs are different, similarity scores slightly differ across runs.

3.6 Similarity threshold

The similarity threshold that indicates the similarity score above which the predictions are almost 100% correct is dynamically adjusted for the Kaggle dataset, as this dataset contains humanly annotated resume job pairs. The system finds the highest similarity score among wrong predictions and sets the mean between all runs of it as a similarity threshold. Correct jobs are matched with the respective job category, which allows for direct comparison between the predicted and the originally annotated job category. The similarity threshold is set to 0.7 by default, and it is dynamically adjusted to the mean of the highest similarity score that made a wrong prediction across all runs.

The MindDig dataset does not allow for dynamic adjustment of the similarity threshold, as this dataset is not humanly annotated. As the jobs are not pre-matched with the corresponding resumes, setting an accurate threshold in advance is not possible with this setup. The results need to be checked by a human annotator in another study, and the threshold needs to be decided after manual inspection of the resume job matches that the system produced. Due to time constraints, this was not done during this research for the MindDig dataset.

3.7 Method limitations

The method has several limitations, which are mainly due to time constraints. The models are not fine-tuned. The dataset training split could be used for fine-tuning, and the validation set could be utilized for hyperparameter tuning (e.g., similarity score).

The validity of the job-resume pairs produced by the models using the MindDig dataset needs to be checked by a human annotator, and they need to decide the similarity threshold that indicates correct and wrong matches. Setting an accurate dynamic similarity threshold during the model performance is not possible for this dataset, as there are no pre-matched resume job pairs. The accuracy of the predictions above a certain threshold can be determined only after this threshold has been set by the human annotator.

4. Results

4.1 Result summary

The performance of the models is evaluated using the average (mean) cosine similarity score of the jobs and resume embeddings of all five runs. Standard deviation is also used to show how much the average similarity varies between runs. The table below shows the scores of all models using the Kaggle and MindDig datasets (Table 1). All 14 resume files and all 21 job

category files provided by MindDig were tested, and the results are listed in columns “MindDig Mean” and “MindDig SD”. The evaluation results from the Kaggle dataset are shared in columns “Kaggle Mean” and “Kaggle SD”.

The highest mean cosine similarity scores were generated by sentence-t5-base (0.768 mean on MindDig and 0.765 on Kaggle) and text-embedding-ada-002 (0.759 on MindDig and 0.771 on Kaggle) models, followed by gtr-t5-base. The next set of models: all-MiniLM-L6-v2, all-MiniLM-L12-v2, multi-qa-MiniLM-L6-cos-v1, multi-qa-distilbert-cos-v1, and all-distilroberta-v1 displayed moderate performance, followed by all-mpnet-base-v2 and the paraphrase-mpnet-base-v2, which scored lower. The worst performing model of all was the TF-IDF, which yielded a similarity score of only 0.008 on the MindDig dataset and 0.035 on the Kaggle dataset.

Models	MindDig Mean	MindDig SD	Kaggle Mean	Kaggle SD
Base1: sentence-t5-base	0.768	0.001	0.765	0.003
Base2: text-embedding-ada-002	0.759	0.001	0.771	0.003
all-MiniLM-L6-v2	0.206	0.005	0.204	0.013
all-MiniLM-L12-v2	0.248	0.007	0.247	0.013
multi-qa-MiniLM-L6-cos-v1	0.186	0.003	0.171	0.017
multi-qa-distilbert-cos-v1	0.192	0.004	0.228	0.007
all-mpnet-base-v2	0.251	0.005	0.256	0.013
all-distilroberta-v1	0.195	0.004	0.14	0.012
paraphrase-mpnet-base-v2	0.213	0.005	0.189	0.014
gtr-t5-base	0.535	0.003	0.523	0.011
TF-IDF	0.008	0.001	0.035	0.003

Table 1. Average (mean) cosine similarity score between job resume embeddings and the Standard deviation of the mean across the runs.

4.2 Qualitative results (correct prediction examples)

The examples below show correctly predicted resume-job pairs generated using the Sentence embedding model “text-embedding-ada-002” on the Kaggle dataset. The resume row contains only a part of the resume due to space limitations, but the entire text is available in Appendix D.

Example 1:

Similarity score: 0.869

Resume: Network and Security Engineer - Capita, Skill Details Network Security- Exprience - 72 months...

Category (from the original dataset): Network Security Engineer

Category (predicted): Network Security Engineer

Example 2:

Similarity score: 0.867

Resume: Java Developer - Vertical Software Expertise in design and development of web applications using J2EE, Servlets, JSP, JavaScript, HTML, CSS, JQUERY, AJAX, JSON....

Category (from the original dataset): Java Developer

Category (predicted): Java Developer

Example 3:

Similarity score: 0.861

Resume: Responsible for analyzing, designing and developing ETL strategies and processes, writing ETL specifications...

Category (from the original dataset): ETL Developer

Category (predicted): ETL Developer

4.3 Error analysis (incorrect prediction examples)

The examples below show incorrectly predicted resume-job pairs generated using the Sentence embedding model “text-embedding-ada-002” on the Kaggle dataset. The resume row contains only a part of the resume due to space limitations, but the entire text is available in Appendix E.

Example 1:

Similarity score: 0.703

Resume: Education Details MBA ACN College of engineering & mgt HR Skill Details Company Details company - HR

Category (from the original dataset): HR

Category (predicted): Blockchain

Example 2:

Similarity score: 0.709

Resume (before preprocessing): Sr. Network Engineer - Cloudatix Network Pvt. Ltd Skill Details CISCO- Exprience - 43 months DHCP- Exprience - 43 months...

Category (from the original dataset): Network Security Engineer

Category (predicted): Health and fitness

Example 3:

Similarity score: 0.712

Resume: Education Details LLB. Dibrugarh University

Advocate Skill Details Legal.- Exprience - Less than 1 year months...

Category (from the original dataset): Advocate

Category (predicted): Health and fitness

5. Discussion

5.1 Interpretation of Results

The 11 models used in this research were chosen to compare the effectiveness of modern embedding models against traditional keyword matching algorithms such as the TF-IDF. The aim was to pick the most robust and highest-performing models as baselines for this study. Results showed that the sentence-t5-base and the text-embedding-ada-002 models generated the highest similarity scores, demonstrating low standard deviation and consistent performance across datasets. The study showed which machine learning techniques provide the most effective matching between candidate profiles and job advertisements. Different models and datasets were used to show under what circumstances do certain techniques outperform others. The next three sections below (5.1.1, 5.1.2, and 5.1.3) focus on describing the effectiveness of each model.

5.1.1 High-performance models

1. sentence-t5-base (MindDig 0.768 / Kaggle 0.765)
2. gtr-t5-base (MindDig 0.535 / Kaggle 0.523)
3. Semantic embedding model text-embedding-ada-002 (MindDig 0.759 / Kaggle 0.771)

The sequence-to-sequence transformer models in this research - "sentence-t5-base", and the encoder-only embedding model - "text-embedding-ada-002", generated the highest cosine similarity scores, which is why they were chosen as base models. Gtr-t5-base ranked third in terms of results. The sequence-to-sequence (encoder-decoder) transformer models are trained on sentence similarity and semantic tasks, making them highly effective in recognizing semantically similar text. Unlike the seq2seq model (which has an encoder and a decoder), Ada-002 (text-embedding-ada-002) only contains an encoder. The encoder converts input text into a numerical representation (a vector) that captures meaning. By omitting the decoder, the model focuses entirely on understanding text instead of generating it. The Ada-002 was trained to place semantically similar sentences close together in the vector space, even if they do not contain an exact keyword overlap. These two models' architectures achieve both high and similar mean similarity scores (~0.76) on both the MindDig and Kaggle datasets, even though they differ in vocabulary and topics, demonstrating robust and consistent performance.

This indicates that the models perform consistently well across different datasets or conditions. The low standard deviations (SD) (0.001–0.003) show that the models’ results don’t fluctuate much across different splits and samples, which suggests that they are stable, reliable, and less sensitive to domain shifts or noise.

All of the Transformer and semantic sentence embeddings (including the low to moderate performance ones below) capture semantic similarities in the job resume pairs as they map sentences into a dense semantic space where cosine similarity reflects semantic closeness. Unlike TF-IDF, they do not match only exact word overlap in resume-job pairs. This results in more bell-shaped distributions, centered around higher scores, because semantically related sentences cluster closer together. Fig. 4 below shows an example of such a distribution generated by the semantic embedding model “text-embedding-ada-002”. The rest of the similarity scores distributions are shared in Appendix B.

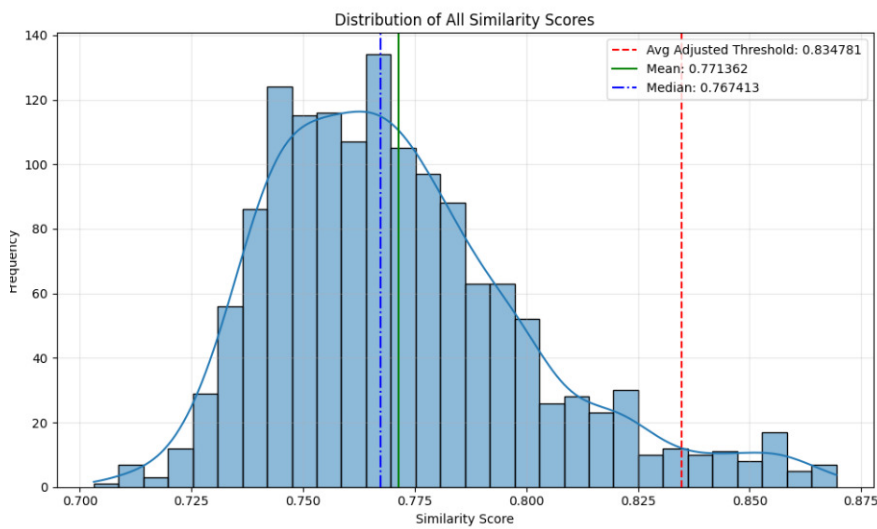


Fig. 4 Distribution of similarity scores with the semantic embedding model text-embedding-ada-002 tested with the Kaggle dataset.

5.1.2 Moderate performance models

1. all-MiniLM-L6-v2 (MindDig 0.206 / Kaggle 0.204)
2. all-MiniLM-L12-v2 (MindDig 0.248 / Kaggle 0.247)
3. multi-qa-MiniLM-L6-cos-v1 (MindDig 0.186 / Kaggle 0.171)
4. multi-qa-distilbert-cos-v1 (MindDig 0.192 / Kaggle 0.228)
5. all-distilroberta-v1 (MindDig 0.195 / Kaggle 0.140)

The next set of models, all-MiniLM-L6-v2, all-MiniLM-L12-v2, multi-qa-MiniLM-L6-cos-v1, multi-qa-distilbert-cos-v1, and all-distilroberta-v1, are smaller, encoder-only, distilled models that are optimized to be efficient at the expense of sacrificing semantic richness. These models are faster and lighter but less accurate. This is visible by their lower mean scores (0.17–0.25) and higher variability indicated by larger SD scores (0.005–0.017) compared to the top-performing models. The higher SD scores suggest that the performance of the models is less stable, and the accuracy can vary across different sample splits or datasets. The difference between similarity scores on the Kaggle and MindDig datasets is more significant on the multi-

qa-distilbert-cos-v1 model, and it spikes from 0.192 to 0.228 on Kaggle. This is because the MindDig dataset contains more domain-specific terms and longer, more complex sentences, which smaller models fail to capture.

5.1.3 Low-performance models

1. all-mpnet-base-v2 (MindDig 0.251 / Kaggle 0.256)
2. paraphrase-mpnet-base-v2 (MindDig 0.213 / Kaggle 0.189)

The all-mpnet-base-v2 and the paraphrase-mpnet-base-v2 are MPNet-based, non-distilled encoder-only models that are trained on paraphrase similarity tasks. They perform moderately well but not as strongly as T5-based models. All-mpnet-base-v2 consistently outperforms paraphrase-mpnet-base-v2 as it is more adapted to general embedding training rather than paraphrase detection. The low SD shows that both models are consistent in their performance, even though they yield mediocre scores.

5.1.4 Very Low performance models

1. TF-IDF (MindDig 0.008 / Kaggle 0.035)

The TF-IDF can not understand semantics and contextual meaning, and captures only exact keyword overlap. This is why it produces near-zero scores for embedding-based semantic tasks, with a mean of 0.008 for MindDig and 0.035 for Kaggle. The small SD values (0.001 for MindDig and 0.003 for Kaggle) indicate that the TF-IDF consistently gives near-zero similarity scores, rather than sometimes performing well and sometimes poorly across runs. The low mean and SD show both ineffectiveness in accurate job-resume comparisons and consistent failure across runs, rather than randomness.

The TF-IDF distribution of all similarity scores in the histogram of Fig. 5 below is heavily skewed to the left, and most scores are close to zero. This shows that most sentence pairs share little or no vocabulary. There's a long, thin tail stretching toward 0.7, showing that a few pairs share many words, so they score higher. The threshold is set to 0.529, which is way above most observations, and only nearly identical strings that share a lot of the same exact words cross it. The TF-IDF histogram is dominated by 0 scores, so unless the wording is almost identical, the similarity is insignificant. The TF-IDF fails to capture semantically similar words with low lexical overlap, such as "Civil site engineer" and "execution of all civil work".

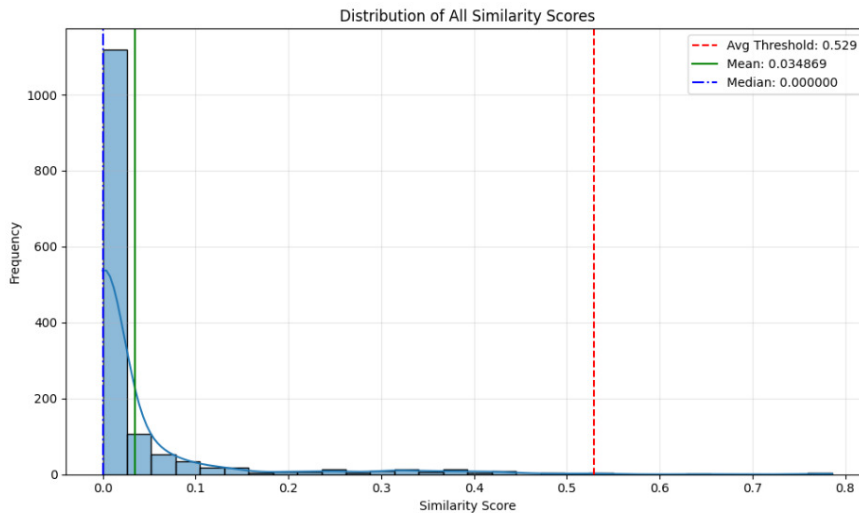


Fig. 5 Distribution of similarity scores with the TF-IDF model tested with the Kaggle dataset.

5.2 Future research

Future research may include using another labeled resume–job pairs dataset and combining it with a fine-tuned Sentence Transformer model with contrastive loss (e.g., SentenceTransformer’s MultipleNegativesRankingLoss), which is expected to boost the performance. Human annotations could also be used for the results produced with the MindDig dataset to determine their accuracy and set a similarity score.

Another future study may combine text-similarity-davinci-001 for text classification of a labeled resume–job pairs dataset, which typically outperforms text-embedding-ada-002 in text classification tasks according to OpenAI test data (OpenAI, 2022a, Fig.6).

Model	Performance
text-embedding-ada-002	90.1
text-similarity-davinci-001	92.2
text-similarity-curie-001	91.5
text-similarity-babbage-001	91.1
text-similarity-ada-001	89.3
Dataset: <u>SentEval</u> (MR, CR, SUBJ, MPQA, SST, TREC, MRPC)	

Fig.6 Text classification comparisons of OpenAI Semantic embedding model (OpenAI, 2022a).

Other ML models could also be a subject of another study in the future, and the results could be tested with more datasets. The results could also be evaluated using other similarity scores such as Euclidean Distance, Manhattan / L1 Distance, or Dot Product. Instead of using raw

similarity scores, the system can also be evaluated in terms of how it ranks the correct job to a given resume using Precision@K, Recall@K, Mean Reciprocal Rank (MRR), or Normalized Discounted Cumulative Gain (nDCG). Accuracy, Precision, Recall, F1 score, and ROC-AUC could also be used with a given threshold, which sets predictions as correct or incorrect.

5.3 Strengths and Limitations of the current study

The current study has focused on comparing Semantic embedding models, Sentence Transformers, and TF-IDF and evaluating their results using cosine similarities. The models were evaluated using only two datasets, and the Sentence Transformers were not fine-tuned due to time constraints. Resume-job pairs of the MindDig dataset were not matched manually with the intention of training the models to work better on the given dataset due to limited resources.

6. Conclusion

The current research focused on studying the effectiveness of 11 distinct techniques for resume job matching. It made a direct comparison between modern ML semantic embedding models and traditional keyword matching algorithms. All models were tested using a public dataset from Kaggle and a private corporate dataset provided by the Swedish MindDig company. The evaluations were run five times per value, and the results were rated using the mean cosine similarity score.

The outcomes showed that the T5-based and Ada models captured the semantic similarities best. These models are more robust across many domains as they are pretrained on large, diverse corpora. The sequence-to-sequence transformer models in this research - “sentence-t5-base” and the encoder-only embedding model “text-embedding-ada-002” generated the highest cosine similarity scores, which is why they were chosen as baseline models. They demonstrated similar similarity scores on both Kaggle and MindDig datasets, indicating that the models perform consistently well across different datasets or conditions. Their low standard deviations suggested that the models are stable, reliable, and less sensitive to domain shifts or noise.

The smaller, distilled models were faster but weaker. The models all-MiniLM-L6-v2, multi-qa-MiniLM-L6-cos-v1, and multi-qa-distilbert-cos-v1 performed poorly due to their lack of domain adaptation, as they were not fine-tuned for specialized content. The TF-IDF model produced the worst results for both datasets as it ignored semantic context and relied entirely on exact keyword overlap. This model performed extremely poorly compared to modern embedding models. This showed that modern ML semantic embedding models significantly outperform traditional keyword matching algorithms.

The research methodology has several limitations that could be a subject of future research. The transformer models could be fine-tuned on the given datasets, so they can fit the data more closely. Other types of ML models and traditional keyword matching algorithms could be compared in the future as well, using different datasets.

7. ChatGPT

I have not used ChatGPT or any AI model to generate text for this thesis.

References

1. Agarwal, T., Kalwani, C., Pathak, D., Sachan, S., Arora, N., & Sahu, S. (2025). OpportuNest – Campus career management system. In *Proceedings of the 3rd International Conference on Disruptive Technologies (ICDT 2025)* (pp. 460–464). Institute of Electrical and Electronics Engineers Inc. <https://doi.org/10.1109/ICDT63985.2025.10986748>
2. Alonso, R., Dessì, D., Meloni, A., & Reforgiato Recupero, D. (2025). A novel approach for job matching and skill recommendation using transformers and the O*NET database. *Big Data Research*, 39, Article 100509. <https://doi.org/10.1016/j.bdr.2025.100509>
3. Bevara, R. V. K., Mannuru, N. R., Karedla, S. P., Lund, B., Xiao, T., Pasem, H., Dronavalli, S. C., & Rupeshkumar, S. (2025). Resume2Vec: Transforming applicant tracking systems with intelligent resume embeddings for precise candidate matching. *Electronics*, 14(4), 794. <https://doi.org/10.3390/electronics14040794>
4. Bian, S., Chen, X., Zhao, W. X., Zhou, K., Hou, Y., Song, Y., Zhang, T., & Wen, J.-R. (2020). Learning to match jobs with resumes from sparse interaction data using multi-view co-teaching network. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM 2020)* (pp. 65–74). ACM. <https://doi.org/10.1145/3340531.3411832>
5. Biol, I. J. A., & Abalorio, C. C. (2024). Utilizing named entity recognition for web-based resume scoring. *International Journal of Engineering Trends and Technology*, 72(7), 381–387. <https://doi.org/10.14445/22315381/IJETT-V72I7P142>
6. Chala, S. A., Ansari, F., Fathi, M., & Tijdens, K. (2018). Semantic matching of job seeker to vacancy: A bidirectional approach. *International Journal of Manpower*, 39(8), 1047–1063. <https://doi.org/10.1108/IJM-10-2018-0331>
7. Dilli Ganesh, V., Erkinjon, T., Alzubaidi, L. H., Vijayakumar, K., Singh, K. R., & Rahuman, A. K. (2025). Revolutionizing talent acquisition in human resources with advanced deep learning-based resume screening and candidate matching algorithms. In *Proceedings of the 2025 3rd IEEE International Conference on Automation and*

- Computation (AUTOCOM 2025) (pp. 1289–1293). IEEE. <https://doi.org/10.1109/AUTOCOM64127.2025.10957186>
8. Dharmendra, T. A., Meenakshi, K., & Bhargava, D. (2025). NLP-Powered Resume Screening and Ranking System. In *ICDT 2025: 3rd International Conference on Disruptive Technologies* (pp. 1361–1366). IEEE. 10.1109/ICDT63985.2025.10986338
 9. Dhobale, P., Bhoir, V., Vyavhare, A., Yelkar, P., & Dharmadhikari, P. A. (2025). ResuMatcher: An intelligent resume ranking system. In *Proceedings of the 3rd International Conference on Intelligent Data Communication Technologies and Internet of Things (IDCIoT 2025)* (pp. 1778–1783). IEEE. <https://doi.org/10.1109/IDCIOT64235.2025.10915179>
 10. Drigas, A., Kouremenos, S., Vrettos, S., Vrettaros, J., & Kouremenos, D. (2004). An expert system for job matching of the unemployed. *Expert Systems with Applications*, 26(2), 217–224. [https://doi.org/10.1016/S0957-4174\(03\)00136-2](https://doi.org/10.1016/S0957-4174(03)00136-2)
 11. Ganesh, D. V., Tullovov, E., Alzubaidi, L. H., Vijayakumar, K., Singh, K. R., & Rahuman, A. K. (2025). Revolutionizing talent acquisition in human resources with advanced deep learning-based resume screening and candidate matching algorithms. In *Proceedings of the 2025 3rd IEEE International Conference on Automation and Computation (AUTOCOM)* (pp. 1289–1293). IEEE. <https://doi.org/10.1109/AUTOCOM64127.2025.10957186>
 12. Gu, X., Jian, L., Rao, C., Bu, Z., & Cheng, X. (2025). Together is better: Knowledge-aware model with resume fusion for online job recommendation. *ACM Transactions on Knowledge Discovery from Data*, 19(3), Article 70. <https://doi.org/10.1145/3716503>
 13. Dutta, G. (2021). Kaggle. *Resume Dataset*. <https://www.kaggle.com/datasets/gauravduttakiit/resume-dataset>
 14. Gautam, G., Sharma, D., & Chaturvedi, V. (2025). Automating talent acquisition assisted by resume parsing system for enhanced job matching. In *Proceedings of the 2025 2nd International Conference on Computational Intelligence, Communication Technology and Networking (CICTN 2025)* (pp. 278–281). Ghaziabad, India: IEEE. 10.1109/CICTN64563.2025.10932337
 15. Jha, R., Paliwal, G., & Jha, B. K. (2025). AI-powered resume builder: Enhancing job applications with artificial intelligence. In *Proceedings of the 2025 3rd International Conference on Disruptive Technologies (ICDT 2025)* (pp. 1655–1660). Greater Noida, India: IEEE. 10.1109/ICDT63985.2025.10986431
 16. Jiechieu, K. F. F., & Tsopze, N. (2020). GitHub. *Skills prediction based on multi-label resume classification using CNN with model predictions explanation*. https://github.com/florex/resume_corpus
 17. Jillani SofTech. (2022). Kaggle. *Updated Resume Dataset*. <https://www.kaggle.com/datasets/jillanisoftech/updated-resume-dataset>
 18. Kamkar, S. A., Sanjay, S., Vaishnavi, P., S & Chaitra, M. (2024). Resume parser and job description matcher. In *2024 8th International Conference on Computational System and Information Technology for Sustainable Solutions (CSITSS)* (pp. 1–6). IEEE. <https://doi.org/10.1109/CSITSS64042.2024.10816751>
 19. Li, P., & Yuan, L. (2025). Optimization of enterprise recruitment strategy using LightGBM decision tree: Improving talent matching accuracy. In *Proceedings of the 2024 5th International Conference on Computer Science and Management*

- Technology (ICCSMT 2024)* (pp. 1022–1027). Association for Computing Machinery. <https://doi.org/10.1145/3708036.3708205>
20. Luhn, H. P. (1958). The automatic creation of literature abstracts. *IBM Journal of Research and Development*, 2(2), 159–165. <https://doi.org/10.1147/rd.22.0159>
 21. Maree, M., & Shehada, W. (2024). Optimizing curriculum vitae concordance: A comparative examination of classical machine learning algorithms and large language model architectures. *AI Switzerland*, 5(3), 1377–1390. 10.3390/ai5030066
 22. Ni, Q. (2022). Deep neural network model construction for digital human resource management with human-job matching. *Computational Intelligence and Neuroscience*, Article 1418020. <https://doi.org/10.1155/2022/1418020>
 23. OpenAI. (2022a, December 15). *New and improved embedding model*. OpenAI. <https://openai.com/index/new-and-improved-embedding-model/>
 24. OpenAI. (2022b, January 25). *Introducing text and code embeddings*. OpenAI. Retrieved from https://openai.com/index/introducing-text-and-code-embeddings/?utm_source=chatgpt.com
 25. Pang, R. (2024). Research on the application of Word2Vec-based job-person matching methods in corporate recruitment. In *Proceedings of the 16th International Conference on Machine Learning and Computing (ICMLC 2024)* (pp. 506–510). ACM. <https://doi.org/10.1145/3651671.3651774>
 26. Pinecone. (2025). *all-MiniLM-L12-v2* [Model documentation]. Pinecone. Retrieved August 30, 2025, from <https://docs.pinecone.io/models/all-MiniLM-L12-v2>
 27. Ramyar, R., Nagarani, G., & Natarajan, S. (2024). Deep learning based approach to streamline resume categorization and ranking. In *Proceedings of 5th International Conference on IoT Based Control Networks and Intelligent Systems (ICICNIS)* (pp. 840–845). IEEE. <https://doi.org/10.1109/ICICNIS64247.2024.10823187>
 28. Reimers, N., & Gurevych, I. (2019, November). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)* (pp. 3982–3992). Association for Computational Linguistics. <https://doi.org/10.18653/v1/D19-1410>
 29. Rezaeipourfarsangi, S., & Milios, E. E. (2023). *AI-powered resume-job matching: A document ranking approach using deep neural networks*. In *Proceedings of the 2023 ACM Symposium on Document Engineering (DocEng 2023)* (Article 3609347). Association for Computing Machinery. <https://doi.org/10.1145/3573128.3609347>
 30. Rosenberger, J., Wolfrum, L., Weinzierl, S., Kraus, M., & Zschech, P. (2025). CareerBERT: Matching resumes to ESCO jobs in a shared embedding space for generic job recommendations. *Expert Systems with Applications*, 275, 127043. <https://doi.org/10.1016/j.eswa.2025.127043>
 31. Salton, G., Wong, A., & Yang, C. S. (1975). A vector space model for automatic indexing. *Communications of the ACM*, 18(11), 613–620. <https://doi.org/10.1145/361219.361220>
 32. Salton, G., & Yang, C. S. (1973). *On the specification of term values in automatic indexing* (Technical Report No. TR 73-173). Department of Computer Science, Cornell University.

33. Sanderson, M., & Croft, W. B. (2012). The history of information retrieval research. *Proceedings of the IEEE*, 100(Special Centennial Issue), 1444–1451. <https://doi.org/10.1109/JPROC.2012.2189916>
34. Sentence Transformers. (2025a). *all-MiniLM-L6-v2* [Model card]. In Hugging Face. Retrieved August 30, 2025, from <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>
35. Sentence Transformers. (2025b). *all-mpnet-base-v2* [Model card]. Hugging Face. Retrieved August 30, 2025, from <https://huggingface.co/sentence-transformers/all-mpnet-base-v2>
36. Sentence Transformers. (2025c). *all-MiniLM-L12-v2* [Model card]. Hugging Face. Retrieved August 30, 2025, from <https://huggingface.co/sentence-transformers/all-MiniLM-L12-v2>
37. Sentence Transformers. (2025d). *multi-qa-MiniLM-L6-cos-v1* [Model card]. Hugging Face. Retrieved August 30, 2025, from <https://huggingface.co/sentence-transformers/multi-qa-MiniLM-L6-cos-v1>
38. Sentence Transformers. (2025e). *multi-qa-distilbert-cos-v1* [Model card]. Hugging Face. Retrieved August 30, 2025, from <https://huggingface.co/sentence-transformers/multi-qa-distilbert-cos-v1>
39. Sentence Transformers. (2025f). *paraphrase-mpnet-base-v2* [Model card]. Hugging Face. Retrieved August 30, 2025, from <https://huggingface.co/sentence-transformers/paraphrase-mpnet-base-v2>
40. Sentence Transformers. (2025g). *sentence-t5-base* [Model card]. Hugging Face. Retrieved August 30, 2025, from <https://huggingface.co/sentence-transformers/sentence-t5-base>
41. Sentence Transformers. (2025h). *all-distilroberta-v1* [Model card]. Hugging Face. Retrieved August 30, 2025, from <https://huggingface.co/sentence-transformers/all-distilroberta-v1>
42. Sentence Transformers. (2025i). *gtr-t5-base* [Model card]. Hugging Face. Retrieved August 30, 2025, from <https://huggingface.co/sentence-transformers/gtr-t5-base>
43. Spärck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1), 11–21. <https://doi.org/10.1108/eb026526>
44. Surya, C. S., Kiriti, G., Saketh, K. S., Charan, P. S., & Anjali, A. (2024). Optimizing job matching through automated resume screening with NLP and machine learning. In *2024 IEEE International Conference on Intelligent Signal Processing and Effective Communication Technologies (INSPECT)* (pp. 1–6). IEEE. <https://doi.org/10.1109/INSPECT63485.2024.10896014>
45. Susan, S., Sharma, M., & Choudhary, G. (2024). Uniqueness meets semantics: A novel semantically meaningful bag-of-words approach for matching resumes to job profiles. *Inteligencia Artificial*, 27(74), 117–132. <https://doi.org/10.4114/intartif.vol27iss74pp117-132>
46. UKP Lab & Hugging Face. (2025). *Sentence Transformers documentation*. Retrieved August 30, 2025, from <https://sbert.net/>
47. Vanetik, N., & Kogan, G. (2023). Job vacancy ranking with sentence embeddings, keywords, and named entities. *Information*, 14(8), 468. <https://doi.org/10.3390/info14080468>

48. Wang, Y. (2025). Research on personalized employment recommendation system for higher vocational students based on deep learning algorithms. *In Proceedings of the 2024 4th International Conference on Big Data, Artificial Intelligence and Risk Management (ICBAR 2024)* (pp. 479–483). Association for Computing Machinery. <https://doi.org/10.1145/3718751.3718828>
49. Yazıcı, M. B., Sabaz, D., & Elmasry, W. (2024). AI-based multimodal resume ranking web application for large scale job recruitment. *In 2024 8th International Artificial Intelligence and Data Processing Symposium (IDAP)* (pp. 1–8). IEEE. <https://doi.org/10.1109/IDAP64064.2024.10710945>
50. Zhang, D. (2024). Research on the AI evaluation and matching system for automatic text recognition, feature information extraction and resume based on artificial intelligence. *In 2024 IEEE 2nd International Conference on Electrical, Automation and Computer Engineering (ICEACE)* (pp. 1043–1048). IEEE. <https://doi.org/10.1109/ICEACE63551.2024.10898204>

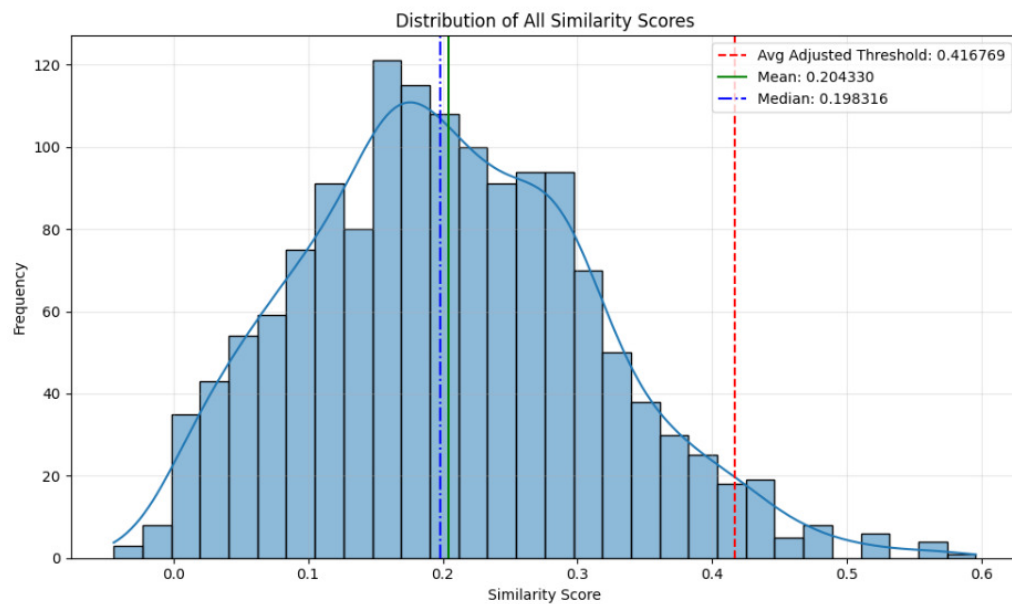
Appendices

Appendix A - Dataset categories

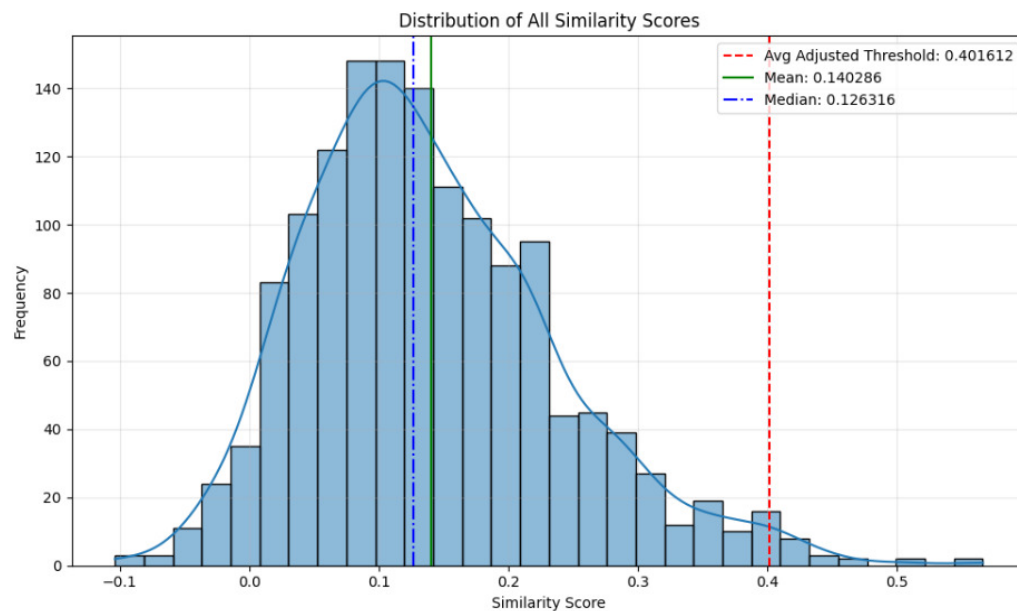
Keggle dataset job fields	MindDig job fields
Data Science	Administration, economics and law.csv
HR	Administration, economics and law_short.csv
Advocate	Body and beauty care.csv
Arts	Building and construction.csv
Web Designing	Crafts.csv
Mechanical Engineer	Culture, media, design.csv
Sales	Data & IT.csv
Health and fitness	Decontamination and sanitation.csv
Civil Engineer	Health care.csv
Java Developer	Hotel, restaurant, catering.csv
Business Analyst	Industrial manufacturing.csv
SAP Developer	Installation, operation and maintenance.csv
Automation Testing	Managers and supervisors.csv
Electrical Engineering	Military occupations.csv
Operations Manager	Natural agriculture.csv
Python Developer	Pedagogical occupations.csv
DevOps Engineer	Sales, purchasing and marketing.csv
Network Security Engineer	Security and surveillance.csv
PMO	Social occupations.csv
Database	Technical occupations.csv
Hadoop	Transport, distribution, storage.csv
ETL Developer	
DotNet Developer	
Blockchain	
Testing	

Appendix B - Distribution of similarity scores

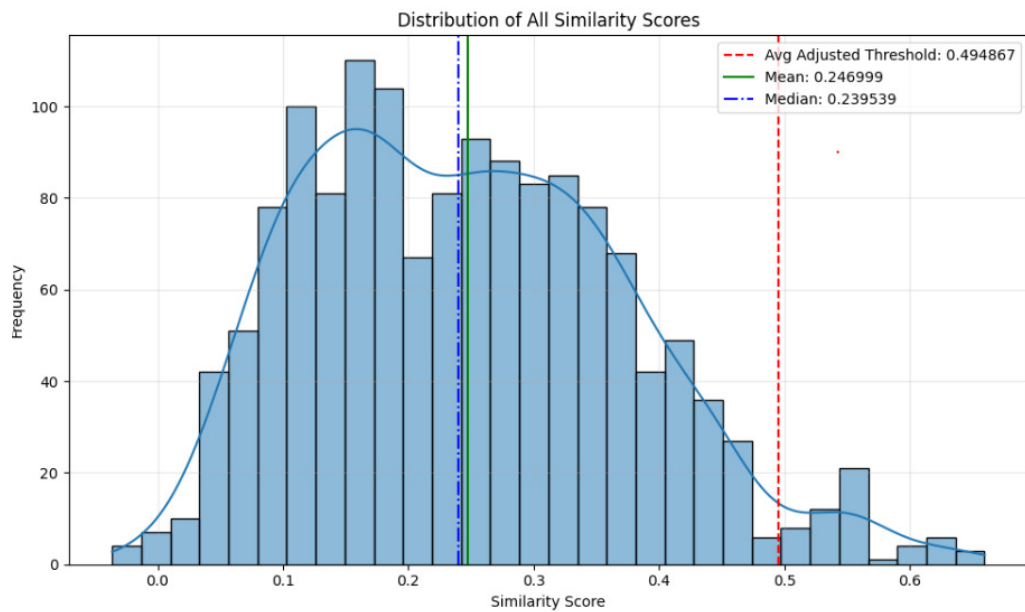
Kaggle dataset



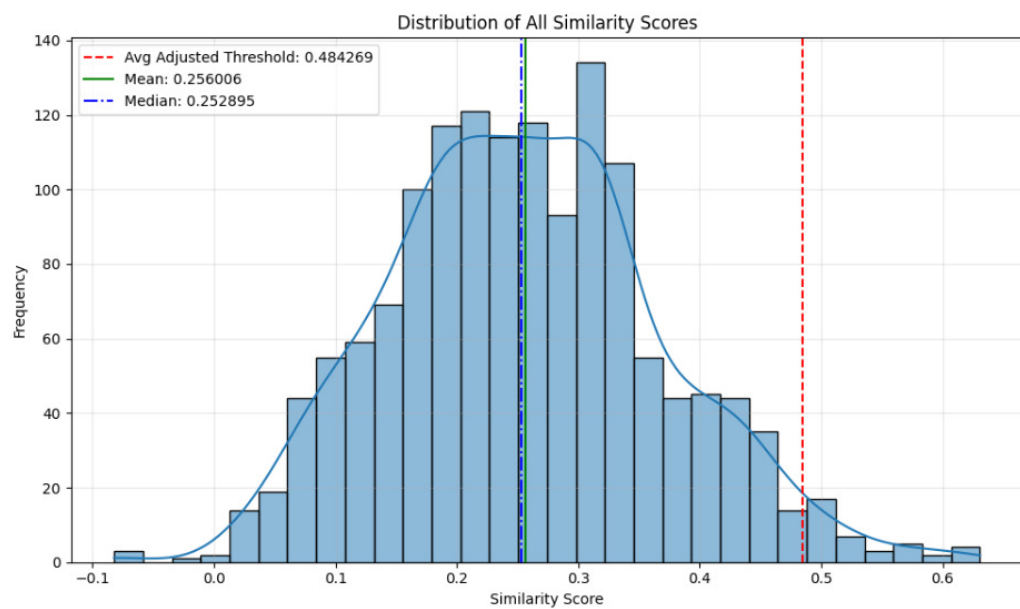
Distribution of similarity scores with the all_MiniLM_L6_v2 model tested with the Kaggle dataset.



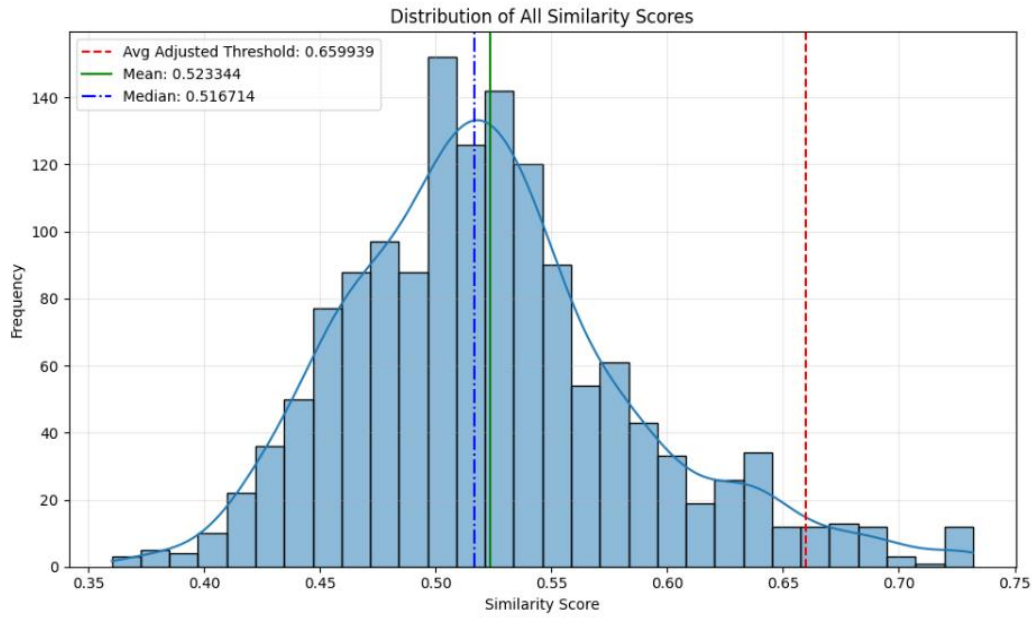
Distribution of similarity scores with the all_distilroberta_v1 model tested with the Kaggle dataset.



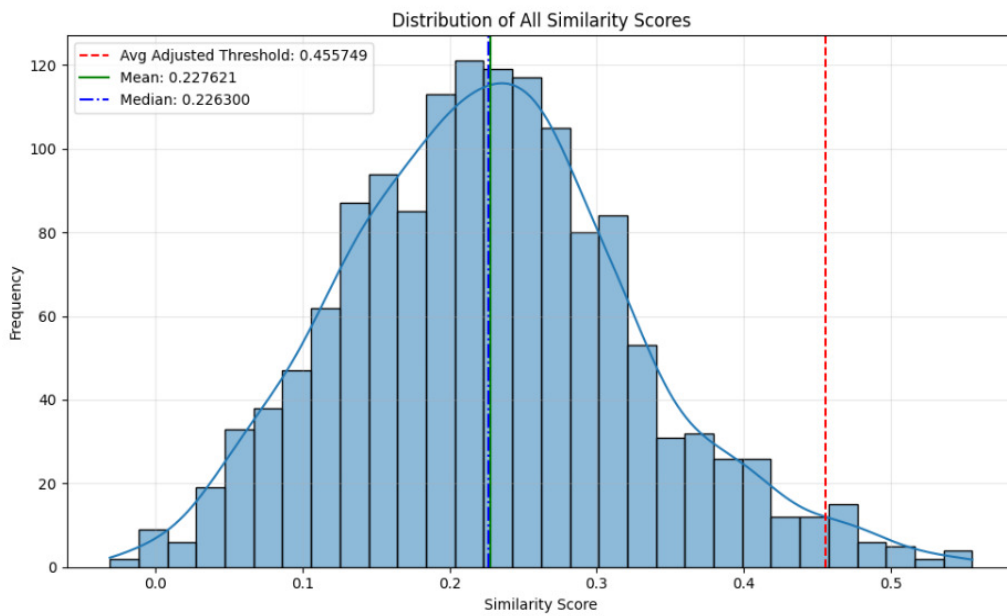
Distribution of similarity scores with the all_MiniLM_L12_v2 model tested with the Kaggle dataset.



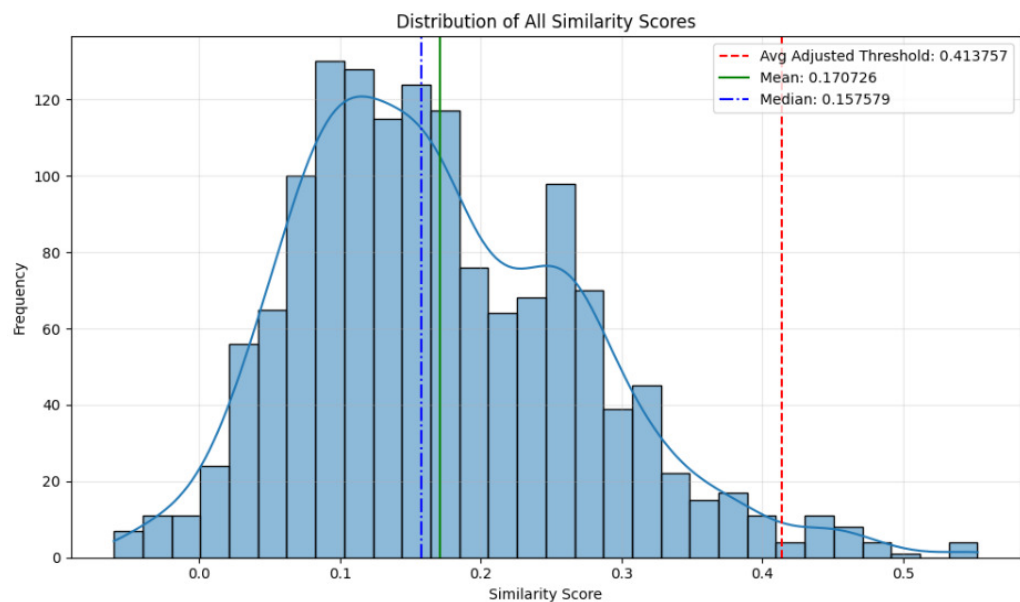
Distribution of similarity scores with the all_mpnet_base_v2 model tested with the Kaggle dataset.



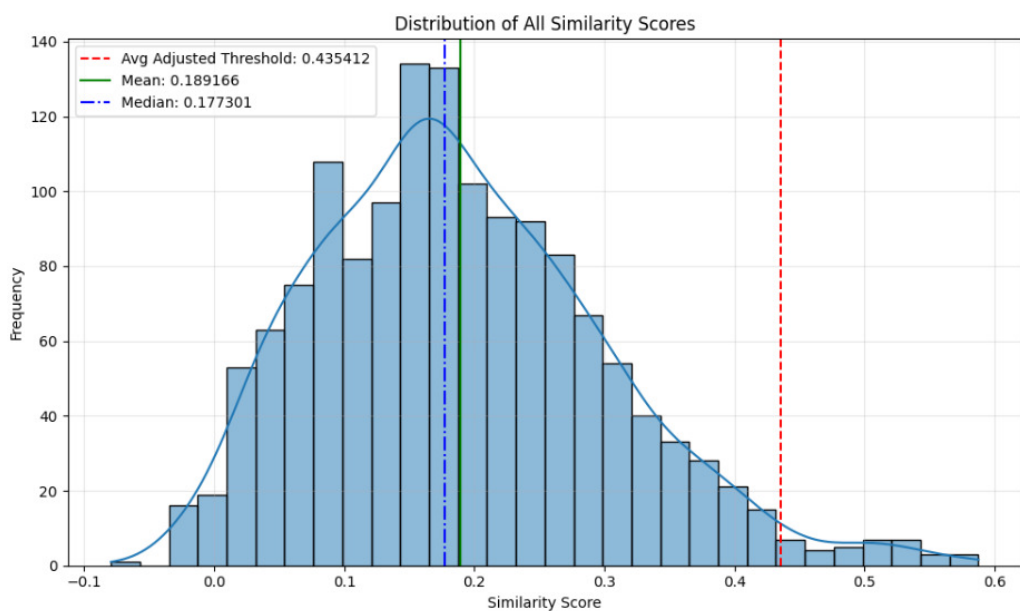
Distribution of similarity scores with the gtr_t5_base model tested with the Kaggle dataset.



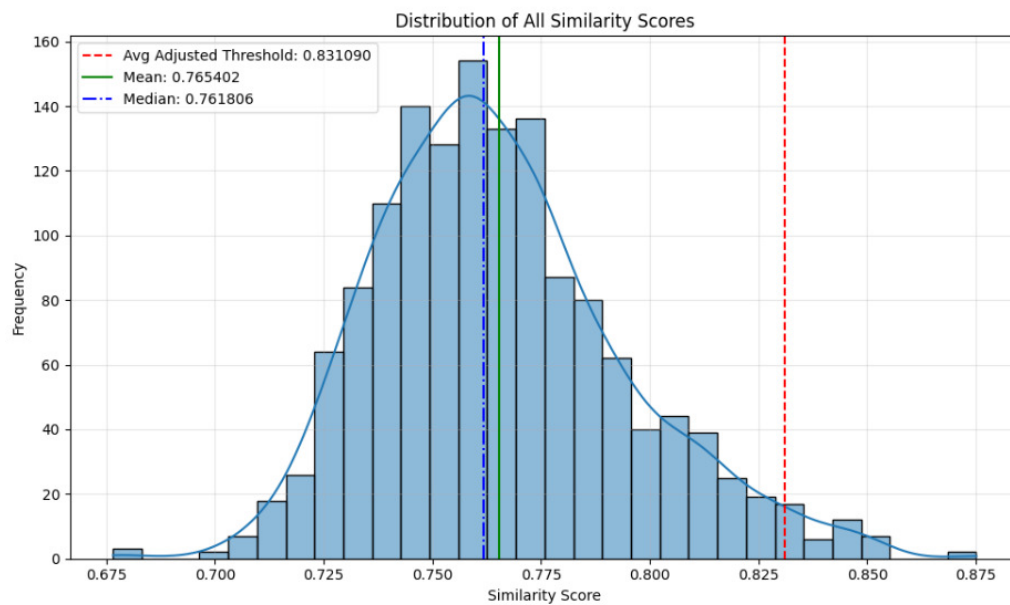
Distribution of similarity scores with the multi_qa_distilbert_cos model tested with the Kaggle dataset.



Distribution of similarity scores with the multi_qa_MiniLM_L6 model tested with the Kaggle dataset.



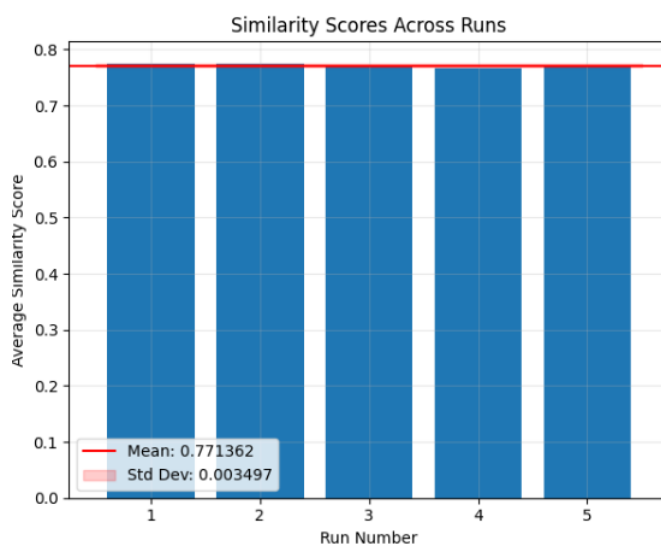
Distribution of similarity scores with the paraphrase_mpnet_base_v2 model tested with the Kaggle dataset.



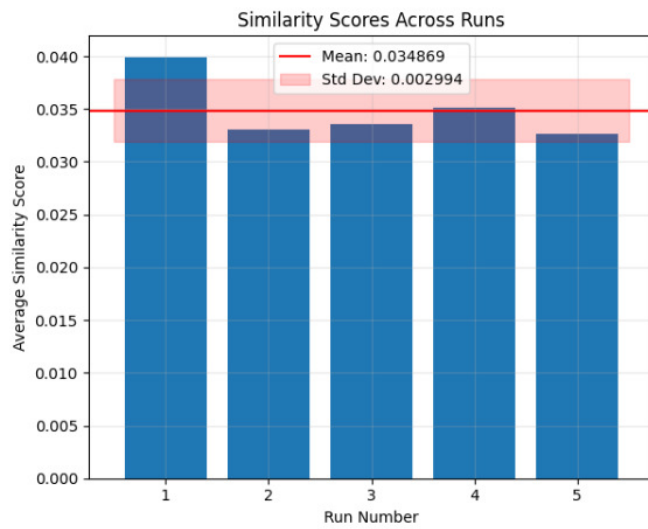
Distribution of similarity scores with the sentence_t5_base model tested with the Kaggle dataset.

Appendix C - Average similarity scores across runs

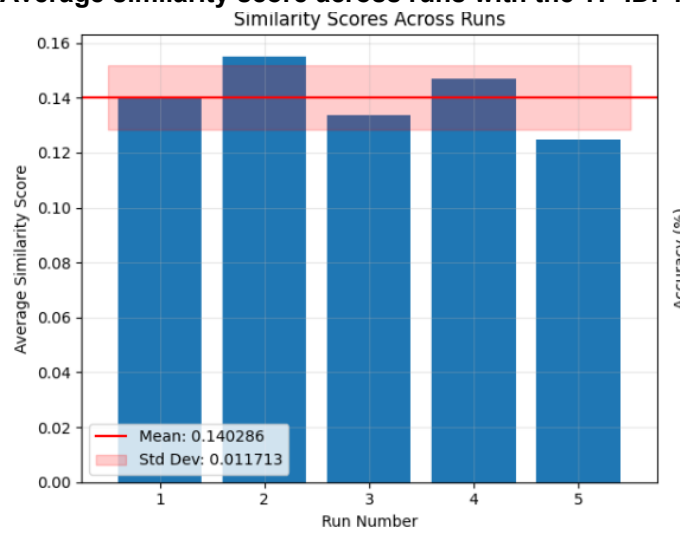
Kaggle dataset



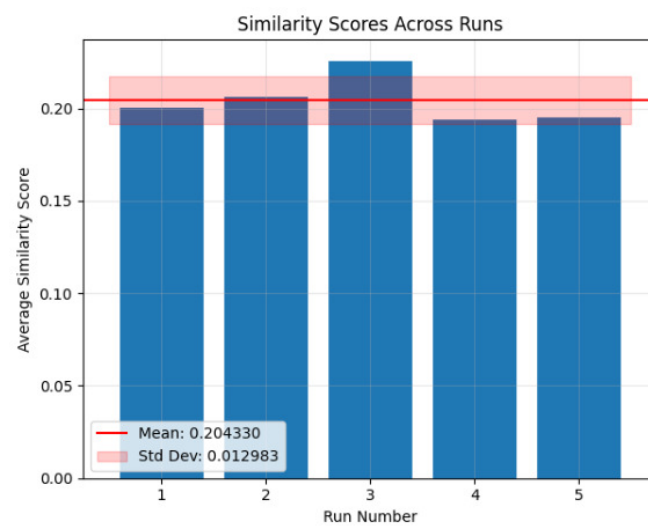
Average similarity score across runs with the Semantic embedding model text-embedding-ada-002 tested with the Kaggle dataset.



Average similarity score across runs with the TF-IDF model tested with the Kaggle dataset.



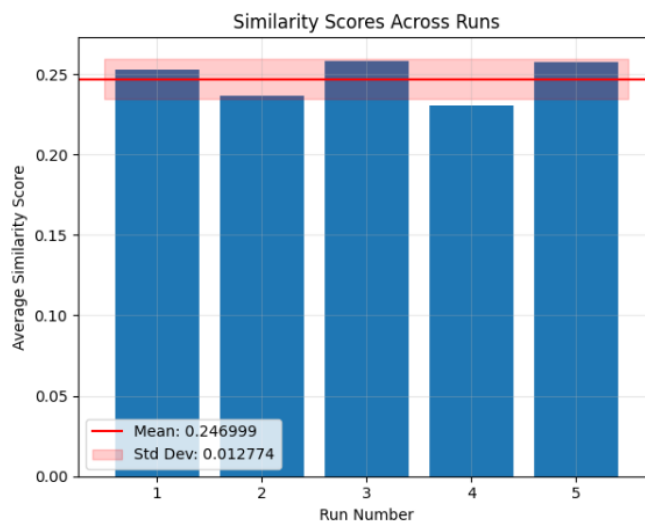
Average similarity score across runs with the all_distilroberta_v1 model tested with the Kaggle dataset.



Average similarity score across runs with the all_MiniLM_L6_v2 model tested with the Kaggle dataset.



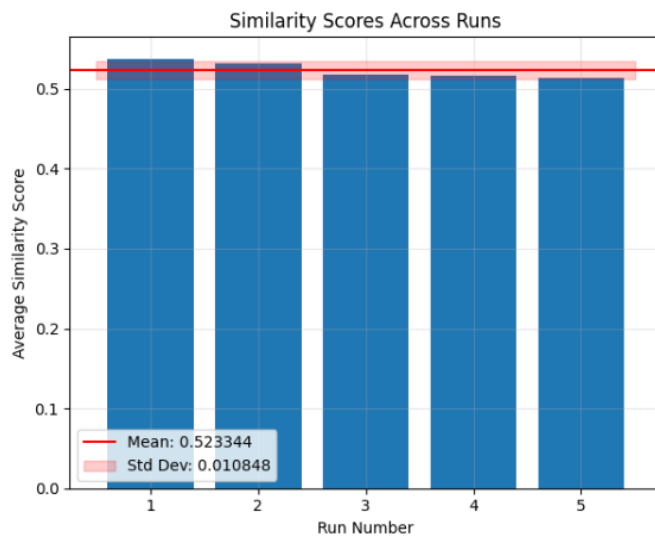
Average similarity score across runs with the all_distilroberta_v1 model tested with the Kaggle dataset.



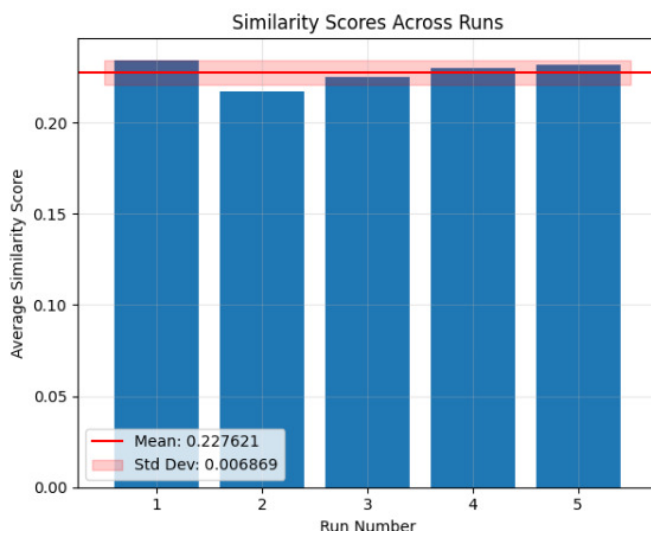
Average similarity score across runs with the all_MiniLM_L12_v2 model tested with the Kaggle dataset.



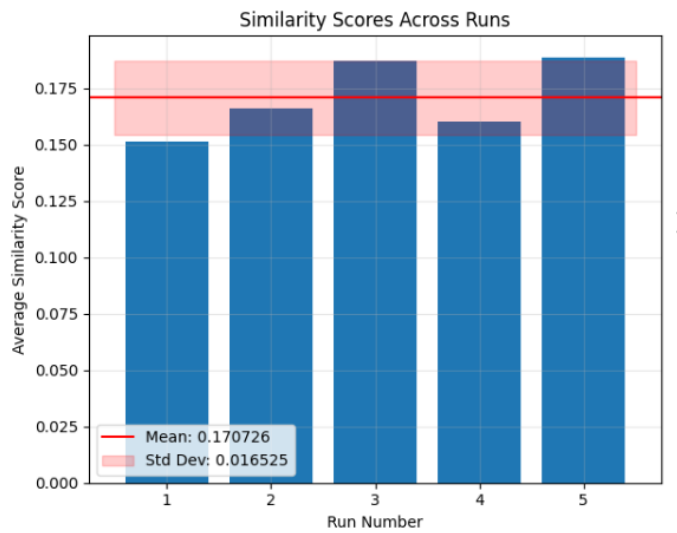
Average similarity score across runs with the all_mpnet_base_v2 model tested with the Kaggle dataset.



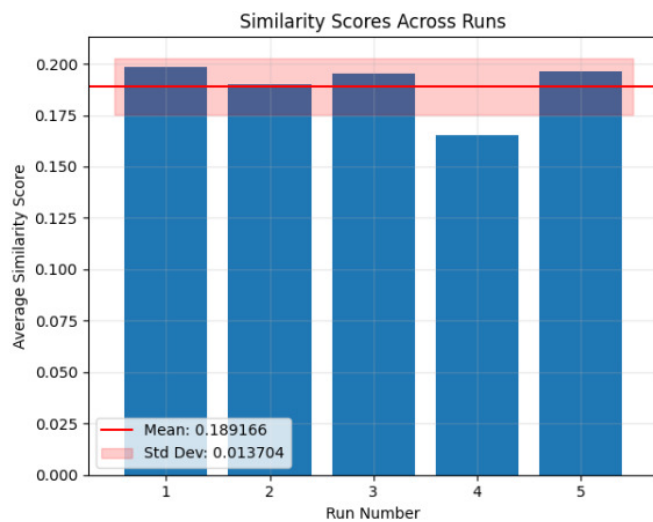
Average similarity score across runs with the gtr_t5_base model tested with the Kaggle dataset.



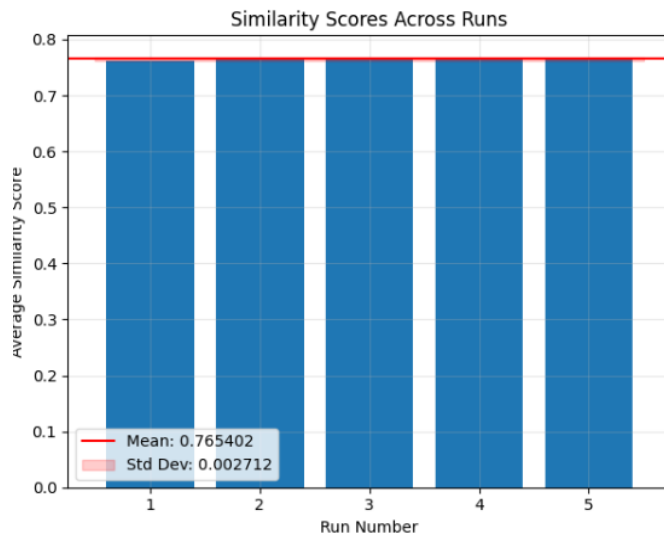
Average similarity score across runs with multi_qa_distilbert_cos_v1 model tested with the Kaggle dataset.



Average similarity score across runs with multi_qa_MiniLM_L6_cos_v1 model tested with the Kaggle dataset.



Average similarity score across runs with paraphrase_mpnet_base_v2 model tested with the Kaggle dataset.



Average similarity score across runs with sentence_t5_base model tested with the Kaggle dataset.

Appendix D - Qualitative results (correct prediction examples)

Correct prediction examples generated by the Sentence embedding model “text-embedding-ada-002” on the Kaggle dataset:

Example 1:

Similarity score: 0.869

Resume (before preprocessing): Skill Set & Experience in Implementing, and troubleshooting network security solutions & Planning and Implementation knowledge of multi vendor firewalls (Cisco ASA, Checkpoint (Upto R.80) Juniper/Netscreen, Fortinet, FWSM) & Familiarity with the latest hardware and network security technologies & Excellent analytical and problem solving skills & Skilled in analyzing and monitoring network security solutions using a variety of Monitoring solutions (Zenoss, Solarwinds, Cisco Prime) & Work Experience on multi client data center environments. & Knowledge and Work experience on Firewall IOS Upgrade projects & Configuration of F5 load balancers, SSL certificate updates, I-Rule. F5 upgrades & Configuration of Cisco Routers (series- 1800, 1900, 2500, 2600, 2800, 3600, Nexus - 5k, 7k) & Configuration of Cisco switches (series - 2960, 3750, catalyst, 4500, 3600) & Working knowledge of Bluecoat Proxy & Knowledge of ITIL process.Education Details September 2006 to August 2011 Bachelor of Engineering (BE) Electronics Pune, Maharashtra A.I.S.S.M.S College of Engineering, University of Pune July 2004 to February 2006 Higher Secondary Certificate Science Pune, Maharashtra Sinhgad College, University Of Pune June 2003 to March 2004 secondary school certificate (SSC) science Pune, Maharashtra M.E.S Boys High School, Maharashtra, Pune Network and Security Engineer

Network and Security Engineer - Capita

Skill Details

Network Security- Exprience - 72 months

CHECKPOINT- Exprience - 72 months

CISCO- Exprience - 72 months

CISCO ASA- Exprience - 72 months

Cisco routing and switching- Exprience - 60 months

Loadbalncing F5- Exprience - 60 months

security- Exprience - Less than 1 year months

Cisco- Exprience - Less than 1 year months

VPN- Exprience - Less than 1 year months

LAN- Exprience - Less than 1 year months

Networking- Exprience - Less than 1 year months

Company Details

company - Capita

description - Work on Client Shared Network and Security infra

• Plan, Implement and troubleshoot customer requests.

• Monitor Datacenter infra 24*7

• Work as On call engineer for weekends to provide Out of office support

company - Capgemini India Pvt. Ltd

description - Part of UK India NOC.

• Work on Client dedicated infra.

• Undergo Client infra handover sessions to streamline client on boarding process

• Act as mentor for juniors.

• Attend Weekly CAAB calls to represent critical Changes.

company - Sungard Availability Services

description - Plan, Troubleshoot and Implement Client network requests

• Project Work - Internet BW upgrade/Downgrade, Decommission

• DR test planning and implementation.

• Setting up L3VPN's for customers

company - SunGard Software Solutions

description - Maintain Client Documentation

• Work on datacenter Remediation Project

• DNS record Management

SunGard Availability Services

Category (from the original dataset): Network Security Engineer

Category (predicted): Network Security Engineer

Example 2:

Similarity score: 0.867

Resume (before preprocessing): Education Details

January 2016 B.E Information Technology Pune, Maharashtra Sawitribai Phule Pune University

Java Developer

Java Developer - Vertical Software

Skill Details

Company Details

company - Vertical Software

description - Expertise in design and development of web applications using J2EE, Servlets JSP, JavaScript, HTML, CSS, JQUERY, AJAX, JSON.

Experienced in developing applications using MVC architecture.

Good understanding of Software Development Life Cycle Phases such as Requirement gathering, analysis, design, development and unit testing.

Languages & open Source Java, J2EE, Spring, Hibernate
Frame Work

Scripting Languages & Server Java JSP, Servlets, DB Connectivity's
Side Program JDBC, JavaScript, jQuery, Ajax, JSON

Application Server TomCat
Database MongoDB, MySql
IDEs Eclipse

1. Project Title: Expense Ledger

Role: Java Developer

Tools and Technologies: Java, Jsp, Servlet, MySql, JavaScript, Json, JQuery, Ajax.

2. Project Title: Trimurti Developer (Realestate)

Role: Java Developer

Tools and Technologies: Java, Jsp, Servlet, MySql, JavaScript, Json, JQuery, Ajax.

3. Project Title: Vimay Enterprise

Role: Java Developer

Tools and Technologies: Java, Jsp, Spring, Hibernate, Maven, JQuery, Ajax.

company - Higher Secondary School

description - Pune, 58.8%

Category (from the original dataset): Java Developer

Category (predicted): Java Developer

Example 3:

Similarity score: 0.861

Resume (before preprocessing): Technical Summary Æ Knowledge of Informatica Power Center (ver. 9.1 and 10) ETL Tool: Mapping designing, usage of multiple transformations. Integration of various data source like SQL Server tables, Flat Files, etc. into target data warehouse. Æ SQL/PLSQL working knowledge on Microsoft SQL server 2010. Æ Unix Work Description: shell scripting, error debugging. Æ Job

scheduling using Autosys, Incident management and Change Requests through Service Now, JIRA, Agile Central & Basic knowledge of Intellimatch (Reconciliation tool) Education Details

January 2010 to January 2014 BTech CSE Sangli, Maharashtra Walchand College of Engineering

October 2009 H.S.C Sangli, Maharashtra Willingdon College

August 2007 S.S.C Achievements Sangli, Maharashtra Martin's English School

ETL Developer

IT Analyst

Skill Details

ETL- Experience - 48 months

EXTRACT, TRANSFORM, AND LOAD- Experience - 48 months

INFORMATICA- Experience - 48 months

MS SQL SERVER- Experience - 48 months

RECONCILIATION- Experience - 48 months

Jira- Experience - 36 months

Company Details

company - Tata Consultancy Services

description - Project Details

Client/Project: Barclays UK London/HEXAD

Environment: Informatica (Power Center), SQL Server, UNIX, Autosys, Intellimatch.

Project Description:

The objective is to implement a strategic technical solution to support the governance and monitoring of break standards - including enhancements to audit capabilities. As a part of this program, the required remediation of source system data feeds involves consolidation of data into standardized feeds.

These remediated data feeds will be consumed by ETL layer. The reconciliation tool is designed to source data from an ETL layer. The data from the Front and Back office systems, together with static data must therefore be delivered to ETL. Here it will be pre-processed and delivered to reconciliation tool before the reconciliation process can be performed.

Role and Responsibilities:

& Responsible for analyzing, designing and developing ETL strategies and processes, writing ETL specifications

& Requirement gathering

& Making functional documents and low level documents

& Developing and debugging the Informatica mappings to resolve bugs, and identify the causes of failures

& User interaction to identify the issues with the data loaded through the application

& Developed mappings using different transformations

company - Tata Consultancy Services

description - Project Details

Client/Project: Barclays UK London/HEXAD

Environment: Informatica (Power Center), SQL Server, UNIX, Autosys, Intellimatch.

Project Description:

The objective is to implement a strategic technical solution to support the governance and monitoring of break standards - including enhancements to audit capabilities. As a part of this program, the required remediation of source system data feeds involves consolidation of data into standardized feeds.

These remediated data feeds will be consumed by ETL layer. The reconciliation tool is designed to source data from an ETL layer. The data from the Front and Back office systems, together with static data must therefore be delivered to ETL. Here it will be pre-processed and delivered to reconciliation tool before the reconciliation process can be performed.

Role and Responsibilities:

- Responsible for analyzing, designing and developing ETL strategies and processes, writing ETL specifications

- Requirement gathering

- Making functional documents and low level documents

- Developing and debugging the Informatica mappings to resolve bugs, and identify the causes of failures

- User interaction to identify the issues with the data loaded through the application

- Developed mappings using different transformations

Category (from the original dataset): ETL Developer

Category (predicted): ETL Developer

Appendix E - Error analysis (incorrect prediction examples)

Incorrect prediction examples generated by the Sentence embedding model “text-embedding-ada-002” on the Kaggle dataset:

Example 1:

Similarity score: 0.703

Resume (before preprocessing): Education Details

MBA ACN College of engineering & mgt

HR

Skill Details

Company Details

company - HR Assistant

description -

Category (from the original dataset): HR

Category (predicted): Blockchain

Example 2:

Similarity score: 0.709

Resume (before preprocessing): Operating Systems: Windows, Linux, Ubuntu Network Technologies : Cisco Routing and Switching, InterVLAN Routing, Dynamic Protocols i.e. RIPv2, RIPv6, OSPF, EIGRP. Static Routing, ACL, VTP, VLAN, EtherChannel, HSRP, STP, IPv6, Lan Troubleshooting, Structured Network Cabling, Cisco Firewall and Fortinet Firewall, RHEL 6 networks. Networking Devices : Cisco Routers: 1800s, 1900s, 2600s, 2900s, 3600s, 3800s, 7200s etc. : Cisco Switches: Cisco Catalyst 2900s, 3700s, 4850s etc. : HP ProLiant Servers, Dell Server, Lenovo Servers etc. : Fortinet Firewall, Modem etc. Server Technologies : AD, RODC, FTP, Print Server, SCCM, WDS, DHCP, Group Policies, DHCP Server, DNS Server, RIS Server, User policies, computer policies etc. Backup Technologies: Server 2016 backup tools, Symantec Backup EXEC 12D etc. Virtualisation: VMWare ESXi, VMware Workstation, Oracle VirtualBox, GNS3 Network Simulator etc. Education Details

MSC AISECT university in distance education

B. Com Jiwaji University

Sr. Network Engineer

Sr. Network Engineer - Cloudatax Network Pvt. Ltd

Skill Details

CISCO- Experience - 43 months

DHCP- Experience - 43 months

DNS- Experience - 43 months

FTP- Experience - 43 months

LAN- Experience - 43 months

Company Details

company - Cloudatax Network Pvt. Ltd

description - 18.

Project Routers, Switches and Server Configuration with Group Policies

Client Railwire/Railtel Corporation of India Ltd., Mumbai

Type of Project Configuration of WiFi AP of Railway Station in Maharashtra State

Role Sr. Network Engineer

Roles & responsibilities

• Earthing, trenching for equipment.

• Making plans for preparing station to WiFi environment including cabling, installation, commissioning, handing over etc.

• Implementing and maintaining backup schedules.

• Upgrading and backups of Cisco router configuration files.

• Installation, Configuration and Administration of Windows Servers 2008 R2, 2012 R2, 2016 Active Directory, FTP, DNS, DHCP, TFTP.

• Troubleshooting LAN & WAN infrastructure.

• Settings of the networking devices (Cisco Router, switches) co-coordinating with the system/network administrator during implementation.

• Regular Health Checking of File Servers & Application Servers for Security violation, Disk Spaces etc.

• Maintaining & Management, windows 7 & 10 based network.

• Installing & Configuring Network Printers.

Assigning proper rights on shared drives.
 User & Groups management and assigning rights according to organization requirements.
 company - Asha MMPC
 description - Project Routers, Switches and Server Configuration with Group Policies
 Client Asha MMPC, Bali, Pali, Rajasthan
 Type of Project Configuration DPMCU, Milknet Server, EMS Server, QMS Server etc.
 Role IT Executive
 Roles & responsibilities
 Implementing and maintaining backup schedules.
 Upgrading and backups of Cisco router configuration files.
 Installation, Configuration and Administration of Windows Servers 2008 R2, 2012 R2, 2016 Active Directory, FTP, DNS, DHCP, TFTP.
 Troubleshooting LAN & WAN infrastructure.
 Settings of the networking devices (Cisco Router, switches) co-coordinating with the system/network administrator during implementation.
 Regular Health Checking of File Servers & Application Servers for Security violation, Disk Spaces etc.
 Maintaining & Management, windows 7 & 10 based network.
 Installing & Configuring Network Printers.
 Assigning proper rights on shared drives.
 User & Groups management and assigning rights according to organization requirements.
 company - Interface Techno System
 description - Project Routers, Switches and Server Configuration with Group Policies
 Client The Scindia School, Fort, Gwalior (MP)
 Type of Project Configuration and Implementation
 Role Network Engineer
 Roles & responsibilities
 Implementing and maintaining backup schedules.
 Upgrading and backups of Cisco router configuration files.
 Installation, Configuration and Administration of Windows Servers 2000/2003, 2008 R2, 2012 R2, Active Directory, FTP, DNS, DHCP, TFTP, Linux OS.
 Working on troubleshooting of complex LAN infrastructure.
 Settings of the networking devices (Cisco Router, switches) co-coordinating with the system/Network administrator during implementation.
 Regular Health Checking of File Servers & Application Servers for Security violation, Disk Spaces etc.
 Maintaining & Management, windows 7 professional based network.
 Installing & Configuring Network Printers.
 Installing Server side Software's & Utilities.
 Assigning proper rights on shared drives.
 Managing user account
 company - Interface Techno System
 description - Project Network Implementation and Configuration
 Client M/s Teva API India Pvt Ltd, Bhind (MP)

Type of Project Implementation and Configuration

Role Network Support Engineer

Roles & responsibilities

• Implementing and maintaining backup schedules.

• Upgrading and backups of Cisco router configuration files.

• Installation, Configuration and Administration of Windows Servers 2000/2003, 2008 R2, 2012 R2, Active Directory, FTP, DNS, DHCP, TFTP, Linux OS.

• Working on troubleshooting of complex LAN infrastructure.

• Settings of the networking devices (Cisco Router, switches) co-coordinating with the system/Network administrator during implementation.

• Configuration, Maintenance, check Point Endpoint Security MI Management Console for Laptop Users

• Regular Health Checking of File Servers & Application Servers for Security violation, Disk Spaces etc.

• Logging & Monitoring Calls using CA Unicenter Service Desk Manager.

• Maintaining & Management, windows XP professional based network.

• Installation, Configuration & troubleshooting of Microsoft Outlook.

• Installing & Configuring Network Printers.

• Installing Server side Software's & Utilities.

• Assigning proper rights on shared drives.

• Managing user account

• Issue E Token for laptop user

• Managing more than 350 Desktops and Laptops

• Taking differential and full backups of the server through Symantec Backup Exec 12D

• Restoring the Files, which got corrupted or deleted accidentally.

• Handling escalations of desktop calls from remote locations.

• Monthly/Weekly Report Generations for the servers.

• Preparing MIS, vendor & inventory management reports.

company - Nava Bharat Press (P) Ltd

description - Role System Engineer

Roles & responsibilities

• Server Management

• Backup management

• Helpdesk, SLA monitoring and incident management

• Hosting client meeting at scheduled interval

• System Management and Troubleshooting

• Hardware Troubleshooting

Additional Qualification & Certifications

• Manual and Computerized accounting with Tally ERP9. Tally certification from Tally Academy.

Category (from the original dataset): Network Security Engineer

Category (predicted): Health and fitness

Example 3:

Similarity score: 0.712

Resume (before preprocessing): Education Details

LLB. Dibrugarh University

Advocate

Skill Details

Legal.- Exprience - Less than 1 year monthsCompany Details

company - Legal.

description - Advocate

Category (from the original dataset): Advocate

Category (predicted): Health and fitness